

Less Is More: Failing Test Generation with Large Language Models

TSZ-ON LI, The Hong Kong University of Science and Technology, China

XIAOCHUAN YAN, Northeastern University, China

WENXI ZONG, Northeastern University, China

YIBO WANG, Northeastern University, China

HAOYE TIAN, Aalto University, Finland

CONGYING XU, The Hong Kong University of Science and Technology, China

YING WANG*, Northeastern University, China

SHING-CHI CHEUNG*, The Hong Kong University of Science and Technology, China

JEFF KRAMER, Imperial College London, United Kingdom

Failing test generation is challenging. It involves searching in a vast space for fault-triggering test inputs and the oracles asserting these faulty executions. Despite techniques proposed to generate tests using large language models (LLMs), they are ineffective in finding failing tests, particularly for programs that implement non-trivial coding tasks such as medium/advanced-level coding contest problems.

To tackle this limitation, we are inspired by an earlier finding that constituent snippets within a program typically implement simpler coding tasks compared to the program as a whole. As a result, LLMs can be leveraged to generate failing tests that target a program's constituent snippets, thereby revealing the program defects.

Leveraging this insight, we propose Microscopic Test Generation (MITGEN), a novel technique of failing test generation. Unlike previous approaches that generate tests to fulfill code coverage, MITGEN focuses on generating tests that reveal faults in a given program's constituent code snippets. We evaluate MITGEN using Starcoder2-15B-instruct-v0.1, Meta-Llama-3-8B-Instruct and CodeQwen1.5-7B-Chat, on two popular benchmarks (EvoEval-Difficult and ClassEval) and 100 real-world subjects. We compare MITGEN with three baselines, including state-of-the-art approaches (Differential Prompting and Pynguin) in finding failing tests. The evaluation results show that MITGEN's recall is 0.66, 112.7% enhancement over the best baseline (0.31).

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**;

Additional Key Words and Phrases: failure-revealing test cases, large language models, code infilling, code correctness assessment

*Corresponding authors.

Authors' addresses: Tsz-On Li, tol@connect.ust.hk, The Hong Kong University of Science and Technology, China; Xiaochuan Yan, yanxiaochuan96@outlook.com, Northeastern University, China; Wenxi Zong, iamwenxiz@163.com, Northeastern University, China; Yibo Wang, yibowangcz@outlook.com, Northeastern University, China; Haoye Tian, haoye.tian@aalto.fi, Aalto University, Finland; Congying Xu, cxubl@connect.ust.hk, The Hong Kong University of Science and Technology, China; Ying Wang*, wangying@swc.neu.edu.cn, Northeastern University, China; Shing-Chi Cheung*, scc@cse.ust.hk, The Hong Kong University of Science and Technology, China; Jeff Kramer, j.kramer@imperial.ac.uk, Imperial College London, United Kingdom.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/2-ART

<https://doi.org/10.1145/3793675>

1 Introduction

Failing test generation involves two challenges. First, it requires searching within a vast input space for fault-triggering test inputs [27]. Second, it requires constructing a test oracle that can precisely assert the occurrence of program failures. Indeed, test oracle construction is generally an undecidable problem [42].

Conventional test generation approaches like Evosuite [26] and Randoop [74] target mostly the detection of regression faults, which occur when a program is not backward compatible after code updates. These approaches assume the program-under-test is correct [20]. Therefore, test oracles can be constructed based on the test execution results of the program-under-test (PUT).

To extend detection beyond regression faults, recent approaches [24, 52, 86, 119] have leveraged Large Language Models (LLMs) for test generation without assuming the program correctness. For example, ChatTester [119] proposes prompting strategies for an LLM to generate unit tests from scratch. Differential Prompting [52] combines LLM with differential testing to generate failing tests. Specifically, it first leverages an LLM to generate alternative implementations for a PUT based on its description. Then, it conducts differential testing between these alternative implementations and the PUT to generate inconsistency-revealing test cases, which are treated as failing tests.

However, these approaches are ineffective in generating failing tests. For example, only 41.4% of the failing tests generated by Differential Prompting [52] for Codeforces subjects have true fault-triggering test inputs and effective assertions. Only 41.0% of the compilable unit tests generated by ChatTester [119] for open-source projects have effective assertions.

One possible reason is that an LLM’s performance in test generation for a coding task can be jeopardized by the complexity of the task. Specifically, recent studies [45, 101] point out that generating test cases for complex coding tasks requires LLMs to perform sophisticated reasoning in comprehending program logic and path constraints. However, LLMs are intrinsically weak at reasoning [25, 77]¹. For instance, Differential Prompting’s success rate in generating failing tests from faulty programs that implement advanced-level Codeforces problems is merely 22.5%, much lower than that on beginning/intermediate-level Codeforces problems (66.7%).

To tackle the above limitation of LLMs, we are inspired by the finding from Ciniselli et al. [13] that real-world programs often comprise small snippets implementing diverse coding tasks. These coding tasks have different levels of complexity. Generally, finer snippets (e.g., a program’s constituent snippets) implement less complex coding tasks than coarser snippets (e.g., the entire program). This suggests that even if a PUT performs a non-trivial coding task, we can still leverage LLMs to effectively generate failing tests that target its constituent snippets, ultimately revealing defects in the PUT.

Figure 1 illustrates a PUT that computes the greatest difference between subarrays of an input array at $O(n)$ time complexity. The overall algorithm is sophisticated and makes it challenging for LLMs to generate tests that directly expose the fault at line 83 (`if subArrayDiff[i] <= min`). However, when given a constituent snippet that performs a relatively simpler task of just computing the greatest difference, LLMs can more effectively generate failing tests targeting this simpler snippet, successfully revealing the fault in the original implementation.

Driven by this insight, we develop Microscopic Test Generation (MITGEN), a novel technique to generate failing tests. Compared to earlier approaches [24, 52, 86, 119] that generate failing tests for an entire PUT, MITGEN generates failing tests for a PUT’s constituent code snippets. Specifically, MITGEN first parses a given PUT into an Abstract Syntax Tree (AST). Following existing works [64, 106, 123] on automatic program repair, MITGEN

¹Despite recent reasoning models (e.g., DeepSeek-R1 [31]) demonstrating better reasoning capability than earlier LLMs, a leaderboard LiveCodeBench [43] shows that reasoning models still have weak coding capability on non-trivial tasks. For instance, for coding contest problems created after DeepSeek-R1’s release date (20th Jan 2025), LiveCodeBench [43] shows that DeepSeek-R1 achieves only 36.7% pass@1 for medium-level problems, and 0.0% pass@1 for hard-level problems. This result indicates that reasoning models can still be weak at reasoning with non-trivial tasks.

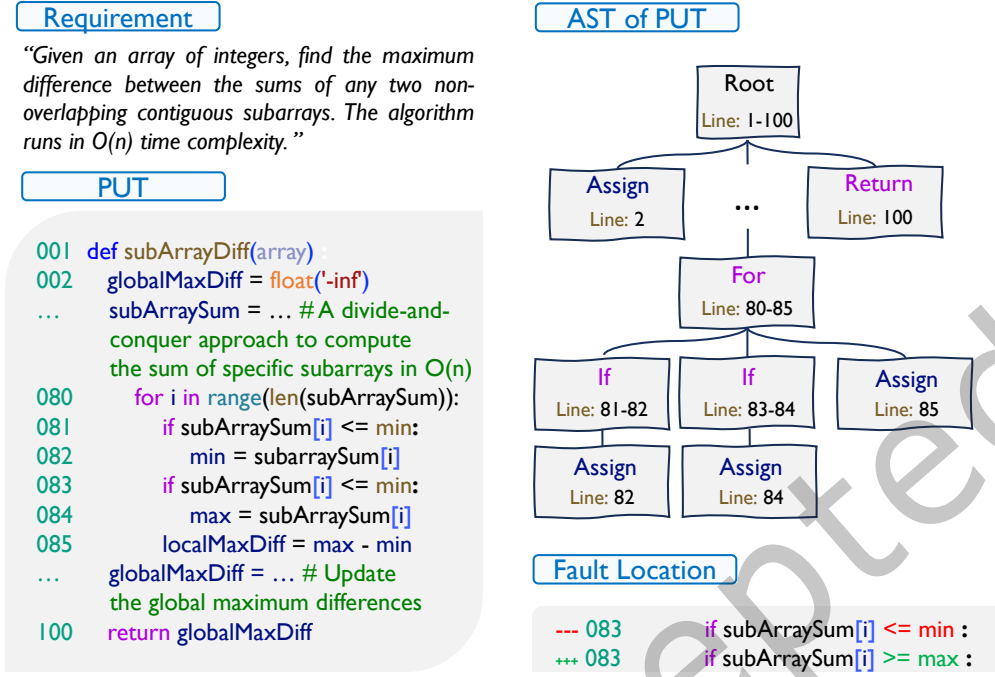


Fig. 1. An example of a faulty program for illustrating the task of generating failing tests. The fault is located at line 83.

assumes that the PUT’s fault(s) are captured by one of the parsed AST nodes. For example, the fault at line 83 of the PUT in Figure 1 is captured by the If-node tagged with “Line: 83-84”. Then, MITGEN leverages LLM to generate failing tests for the code snippets represented by these AST nodes. Specifically, MITGEN extends the test generation stage of Differential Prompting, which leverages an LLM to synthesize two alternative implementations based on the program description of a PUT. Outputs consistently returned by a test on the alternative implementations are considered the expected output. A failing test is found when the expected output differs from the test output by PUT.

A key challenge in realizing MITGEN is identifying those AST nodes for which alternative implementations can be effectively generated. Specifically, an AST node may implement a non-trivial coding task that an LLM is ineffective in generating appropriate alternative implementations. For example, in Figure 1, suppose the AST nodes in line 3 to 79 contribute to a challenging coding task which requires sophisticated logical reasoning to formulate the required algorithm. As recent studies [51, 77, 93, 111] show that LLMs are prone to hallucinations when dealing with logical reasoning, an LLM may synthesize alternative implementations that are misaligned with the requirement of the coding task. These incorrect alternative implementations can mislead the failing test generation, while validating their correctness is non-trivial due to the difficulty of obtaining oracles.

To tackle this challenge, MITGEN exploits a finding made by CAPGEN [106], our earlier work on automatic program repair. Particularly, CAPGEN finds that a faulty snippet and its patch often share similarity in three contexts (namely *Genealogy Contexts*, *Variable Contexts* and *Dependency Contexts*). Leveraging this finding, CAPGEN infers the correctness of a patch by computing the similarity between the patch and the faulty snippet in these three contexts. Inspired by CAPGEN’s finding, MITGEN considers an alternative implementation synthesized by an LLM for an AST node to be a plausible patch for the node. As such, MITGEN infers the likelihood that an alternative implementation is a correct patch, by computing the similarity between the

alternative implementation (a plausible patch) and the original snippet (a potentially faulty snippet) in these three contexts. With this method, MITGEN infers the correctness of alternative implementations synthesized for each AST node, in order to select the AST nodes that an LLM has the highest likelihood to generate correct alternative implementations.

MITGEN incorporates a novel Context-aware Mask Location Prioritization (CamPri) mechanism to select the most likely AST node whose alternative implementation synthesized offers a correct patch if the node is buggy. For each AST node of a PUT, CamPri leverages an LLM to generate multiple (by default, 10) alternative implementations for the code snippet represented by each AST node, as shown in Step 1 of Figure 2. For ease of presentation, we refer to the code snippet, for which alternative implementations are to be generated as “snippet-under-test” (SUT). CamPri first computes the context similarity between them for each pair of SUT and one of its alternative implementations. CamPri then computes the overall context similarity by averaging the ones calculated for the SUT and each of its alternative implementations. If the overall context similarity is high, the alternative implementations likely fix the SUT’s fault according to CAPGEN [106]’s finding. These alternative implementations likely allow MITGEN’s test generation stage to effectively generate failing tests.

It is possible that non-faulty SUT also has high context similarity with its alternative implementations, and thus will be selected to derive failing tests. Fortunately, Differential Prompting’s evaluation [52] shows that its test generation stage (which is adopted by MITGEN) has a low probability of returning failing tests (false alarms) for non-faulty code. Our evaluation result also shows that MITGEN’s rate of returning false alarms for non-faulty programs is merely 0.15.

We implement MITGEN with three popular LLMs: Starcoder2-15B-instruct-v0.1 [59], Meta-Llama-3-8B-Instruct [68] and CodeQwen1.5-7B-Chat [7]. These local models have been popularly rated across various leaderboards (e.g., Evalplus [55], Bigcodebench-leaderboard [124]) and are evaluated by recent works on code/test generation [45, 72, 102], indicating that these models are representative of real-world LLMs on coding tasks related to MITGEN. We evaluate MITGEN on two code generation benchmarks (EvoEval-Difficult [107], ClassEval [23]) and 100 subjects from real-world projects.

EvoEval-Difficult consists of 100 subjects that implement challenging code tasks (e.g., coding tasks that are composed of less common constraints or requirements [55], as illustrated in §2.1). ClassEval comprises 100 class-level code generation tasks that emulate real-world software development. Both benchmarks comprise non-trivial coding tasks. In addition, these benchmarks were released after the knowledge cut-off date of selected LLMs, and thus mitigate the threat of data contamination.

As these benchmarks are code generation benchmarks that comprise only correct programs, we derive faulty programs from these correct programs by either injecting faults seeded by researchers [69] for a benchmark (if available) or applying an existing fault injection tool [63].

To assess MitGen’s performance on real-world programs, we evaluate MitGen on 100 real-world subjects collected from two sources: (1) SWE-Bench-Live [122], and (2) 164 projects covered in SWE-Bench-Live. SWE-Bench-Live is an issue-resolving benchmark that consists of 1,565 subjects spanning 164 real-world projects, as of 19 August 2025. Each subject is essentially a pull request that addresses a problem described in an issue report. We choose SWE-Bench-Live because, to the best of our knowledge, it is currently the only issue-resolving benchmark that consists of subjects created after the knowledge cut-off dates of LLMs selected in our evaluation (i.e., 1st January 2024). SWE-Bench-Live provides 33 subjects that match our subject selection criteria (described in §4.1.1). We further collect 184 subjects from the 164 projects covered in SWE-Bench-Live that match our subject selection criteria but are not included in SWE-Bench-Live. Then, we randomly select 67 subjects from these 184 subjects, so that the number of real-world subjects in our evaluation is 100 (33+67), aligning with the number of subjects in EvoEval-Difficult and ClassEval.

To apply MITGEN to real-world programs, we address the challenge that documentations describing the program requirements of arbitrary code could be unavailable in real-world programs [1]. Specifically, program

requirements are essential for MITGEN (LLM) to derive alternative implementations. LLM’s performance in code generation could drastically decrease when program requirements are unavailable [49].

To address this challenge, our insight is that program requirements can be derived from contextual information typically available in code commits. Specifically, when developers update project code through code commits, they typically write a message describing the intention of the updated code [39, 40, 112]. Furthermore, developers also bundle passing tests in code commits to validate the functionalities of updated code. These passing tests serve as input-output pairs that illustrate the functionalities of the updated code. The intention of code and illustrative input-output pairs are essential contextual information required by LLM to generate code effectively [29, 49]. Hence, we propose a prompt template to exploit these two pieces of contextual information available in the code-updating commits, allowing MITGEN (LLM) to effectively derive alternative implementations under the absence of explicitly-documented program requirements.

Our evaluation of MITGEN on these two benchmarks shows that MITGEN achieves a recall of 0.66 in generating failing tests, representing a 112.7% improvement over the best baseline (0.31). Meanwhile, MITGEN attains a precision of 0.77, favorably compared to the best baseline (0.65).

Contributions. To summarize, this paper makes the following contributions:

- We propose MITGEN, a novel technique of generating failing tests. In contrast to previous LLM-based test generation approaches [24, 52, 86, 119], which generate test cases for an entire PUT, MITGEN generating failing tests from a PUT’s constituent code snippets. By doing so, MITGEN can potentially overcome a current limitation of LLM that it is weak on non-trivial coding tasks, as observed by existing works [44, 52, 107, 119].
- We propose CamPri, a method that enables MITGEN to prioritize suspicious code snippets. It infers an LLM’s effectiveness in generating correct alternative implementations for a particular code snippet. CamPri exploits a finding made by our earlier work CAPGEN [106] that fault-fixed code and faulty code are often similar in three contexts. Hence, CamPri computes such context similarity to infer an LLM’s effectiveness in generating fault-fixed code (i.e., correct alternative implementations) for a code snippet.
- We implement MITGEN and evaluate it on EvoEval-Difficult, ClassEval and 100 subjects from real-world projects. Our evaluation result shows that MITGEN’s recall in generating failing tests is 0.66, 112.7% enhancement over the best baseline (0.31). Meanwhile, MITGEN’s precision is 0.77, favorably compared to that of the best baseline (0.65). We make our research artifact and experimental data available at <https://mitgent.github.io/>.

1.1 Comparison with our earlier work

MITGEN makes the following new contributions beyond Differential Prompting [52].

- Section 3 presents a novel technique, MITGEN, to generate failing tests. In contrast to Differential Prompting (and other LLM-based test generation approaches [24, 86, 119]) that generates failing tests for an entire PUT, MITGEN derives failing tests from a PUT’s constituent code snippet. This allows MITGEN to overcome the limitation of LLM (and Differential Prompting) on non-trivial coding tasks.
- Section 3.1 presents a new method, CamPri, to select AST nodes that allow failing tests to be effectively derived. CamPri aims at tackling a challenge of realizing MITGEN: an AST node may contribute to a non-trivial coding task, preventing an LLM from effectively deriving failing tests. CamPri was not proposed in [52].
- Section 3.3 discusses the challenge and solution of applying MITGEN to real-world subjects, which is not discussed in [52]. Specifically, MITGEN generates alternative implementations of PUT based on the PUT’s program descriptions, yet program descriptions of arbitrary code may not always be available

in real-world programs. To address this challenge, inspired by existing findings on code commits [49] and LLM-based code generation [29, 94], our insight is that program requirements can be derived from two pieces of contextual information typically available in code commits, namely intention of code and input-output examples. Based on this insight, we propose a prompt template that exploits these two pieces of contextual information in code commits, allowing MITGEN (LLM) to effectively derive alternative implementations under the absence of explicitly-documented program requirements. Our evaluation result shows that MITGEN attains an 201.1% enhancement in recall over the best baseline on real-world subjects.

- Section 4 presents a thorough evaluation of MITGEN, Differential Prompting and three baselines on three dataset (EvoEval-Difficult [107], ClassEval [23] and 100 subjects from real-world projects), which represent more realistic coding scenarios encountered by software developers. Compared to Differential Prompting’s original evaluation dataset, which mainly consists of 40 Quixbugs subjects that implement classical algorithms (e.g., merge sort), EvoEval-Difficult, ClassEval and selected real-world subjects together comprise 300 subjects that implement non-trivial coding tasks [23, 107]. Specifically, the selected real-world subjects enable the assessment of MITGEN/Differential Prompting’s performance in real-world application scenarios. Besides, all three datasets are released after the knowledge cutoff date of selected LLMs, alleviating the data contamination threat. Although Differential Prompting attains good performance on the Quixbugs subjects [52], it struggles to achieve comparable performance on the EvoEval-Difficult, ClassEval and 100 real-world subjects. In comparison, MITGEN achieves a significantly better performance in deriving failing tests and alternative implementations. CamPri offers a new and promising mechanism to prioritize suspicious code snippets. In this submission, we also identify major bottlenecks for further improving MITGEN.

2 Preliminaries

Figure 1 gives an example illustrating the problem of failing test generation. In this example, the PUT intends to compute the greatest difference among subarrays of an array. It contains a predicate fault “if subArrayDiff[i] <= min” at line 83. The correct implementation is “if subArrayDiff[i] >= max”. The fault can result in incorrect outputs. For instance, for input [1, 1, 0, 0], the expected output is 1, but the actual output by the PUT is 0. It is because the fault at line 83 causes max-min (i.e., localMaxDiff) in line 85 to be always 0. Hence, a failing test with the input [1, 1, 0, 0] and an assertion of output 1 can reveal the fault.

Differential Prompting [52] is the state-of-the-art technique proposed for failing test generation. In this section, we illustrate a major limitation of Differential Prompting (§2.1), followed by an introduction of code contexts leveraged by MITGEN to address the limitation (§2.2).

2.1 Differential Prompting

MITGEN tackles the problem of failing test generation for the constituent code snippets of a PUT by extending the test generation stage of Differential Prompting [52]. At the test generation stage, for a given PUT according to its requirements, Differential Prompting generates two alternative implementations, referred to as “*reference versions*”. An output consistently given by both reference versions for a test input is considered the expected output, because consistency in output between reference versions generated by an LLM provides a probabilistic guarantee that the consistent output is correct [97].

Our work further shows that output consistency achieves strong positive correlation with LLM’s correctness (i.e., more consistent outputs indicates that LLM achieves higher correctness in generating programs for a task). This observation is also made in other related works which verify correctness of generated programs via output

consistency [21, 37]. A failing test is found when PUT gives an output different from the expected output for the same test input.

To generate failing tests correctly, a reference version needs to return correct outputs for some fault-revealing inputs. Specifically, successes of failing test generation need to meet **two criteria**:

- **C1:** LLMs can generate two reference versions that fix the faulty statements in the PUT.
- **C2:** Both reference versions can output correct results for some inputs, while they may contain other faults.

Note that to satisfy C1 or C2, the reference versions can be faulty and do not need to return correct outputs for all inputs. Specifically, generating completely correct reference versions can be a challenging task to LLM, as LLMs are prone to faults [88]. Nevertheless, the effectiveness of reference versions in deriving failing tests is not necessarily jeopardized by certain types of faults introduced by LLMs. For instance, reference versions suffer from *missed corner cases*, a dominant fault type (23.53%) in the code generated by LLM [88], still allow failing tests to be successfully derived as long as the missed corner cases do not need to be reached.

Overall, by requiring reference versions to satisfy only C1 and C2, the challenge of generating desired reference versions can be reduced. Specifically, Differential Prompting only needs to generate (search for) reference versions that return correct (consistent) outputs for some inputs, rather than generate reference versions that return correct (consistent) outputs for all inputs (which could be difficult due to the fault-prone nature of LLMs).

Note that an LLM may fail to generate a reference version that fulfills C1 and C2 in some attempts, due to reasons including the stochastic nature of LLM [52]. Specifically, an LLM may generate reference versions that suffer from the same fault as the PUT, making failing tests unable to be derived (i.e., false negative case). In addition, an LLM may generate reference versions that introduce new faults, causing incorrect failing tests to be derived (i.e., false positive case). Manual inspection of false negative/positive cases can incur a significant abuse of human effort [116]. To avoid false negative/positive cases, Differential Prompting relies on the consistency in output returned by two reference versions as an automatic validation mechanism, and iteratively generates pairs of reference versions until both reference versions pass this validation (with at most ten attempts by default [52]). As reported in Differential Prompting’s paper [52], Differential Prompting’s success rate in generating reference versions that fulfill C1 and C2 is 74.6%, 11.0X as the strawman approach (6.8%) that directly generates reference versions (i.e., without any validation).

To further improve Differential Prompting, satisfying C1 and C2 is still a challenging task is often the bottleneck. For instance, suppose that a reference version fixes the fault at line 83 of the PUT in Figure 1, but also introduces faults in the computation of subarrays’ sum (lines 3 and 79). In this case, the reference version may return incorrect outputs for many inputs, making Differential Prompting unsuccessful in finding correct failing tests.

2.2 Context similarities between faulty code and fault-fixed code

As mentioned in §1, MITGEN exploits a finding made by CAPGEN [106] that faulty code and fault-fixed code often have similar contexts, namely *Genealogy Contexts*, *Variable Contexts* and *Dependency Contexts*.

- **Genealogy Context:** Given an AST node N (N is either a Statement node, an Expression node or a Type node), genealogy context of N is defined as the set of node types of N ’s ancestor nodes and sibling nodes in the current function. The similarity of the genealogy context between a faulty node and a fault-fixed node is computed as the proportion of overlapped genealogy context between these two nodes. Specifically, let N_{fault} denote a faulty node and N_{fix} denote a fault-fixed node. Let ϕ_{fault} denote the genealogy context of N_{fault} and ϕ_{fix} denote the genealogy context of N_{fix} . Let t denote a node type, and $\phi_{fault}(t)$ denote the frequency of t in ϕ_{fault} , same for ϕ_{fix} . Let K denote the set of distinct node types in ϕ_{fault} . The equation of computing genealogy context

similarity $f(\phi_{fix}, \phi_{fault})$ is as follows:

$$f(\phi_{N_{fix}}, \phi_{N_{fault}}) = \frac{\sum_{t \in K} \min(\phi_{fix}(t), \phi_{fault}(t))}{\sum_{t \in K} \phi_{fault}(t)}$$

- **Variable Context:** Given an AST node N (N is either a Statement node, an Expression node or a Type node), variable context is defined as the set of <Variable Type, Variable Name> tuple of program variables that are read/write in N . The similarity of variable context between two nodes is computed as the Jaccard distance between their variable context. Specifically, let θ_{fix} denote the variable context of N_{fix} and θ_{fault} denote the variable context of N_{fault} . The equation of computing variable context similarity $g(\theta_{fix}, \theta_{fault})$ is as follows:

$$g(\theta_{fix}, \theta_{fault}) = |\theta_{fix}| * \frac{|\theta_{fix} \cap \theta_{fault}|}{|\theta_{fix} \cup \theta_{fault}|}$$

- **Dependency Context:** Given an AST node N (N is either a Statement node, an Expression node or a Type node), dependency context is defined as the set of node types extracted from AST nodes that have def-use relationship with N . The similarity of dependency context between two nodes is computed as the proportion of overlapped dependency context between these two nodes. Specifically, let ψ_{fix} denote the genealogy context of N_{fix} and ψ_{fault} denote the genealogy context of N_{fault} . Let t denote a node type, and $\psi_{fix}(t)$ denote the frequency of t in ψ_{fix} , same for ψ_{fault} . Let K denote a set of distinct node types in ψ_{fault} . The equation of computing dependency context similarity $f(\psi_{fix}, \psi_{fault})$ is as follows:

$$f(\psi_{N_{fix}}, \psi_{N_{fault}}) = \frac{\sum_{t \in K} \min(\psi_{fix}(t), \psi_{fault}(t))}{\sum_{t \in K} \psi_{fault}(t)}$$

The overall contextual similarity is computed as the multiplication of the three context similarities, i.e., $f(\phi_{N_{fix}}, \phi_{N_{fault}}) * g(\theta_{fix}, \theta_{fault}) * f(\psi_{N_{fix}}, \psi_{N_{fault}})$.

3 Methodology

MITGEN takes a PUT and the corresponding requirement as input, and outputs a failing test if successfully found. Figure 2 illustrates the workflow of MITGEN, consisting of two primary components: Context-aware Mask Location Prioritization (§3.1) and Infilling-based Reference Version Generator (§3.2).

A major challenge in realizing MITGEN is to select appropriate code locations (a.k.a. *masked locations*) for infilling. As discussed in §1, the selected masked locations are expected to implement a coding task that an LLM can generate intended reference versions. We, therefore, propose Context-aware Mask Location Prioritization (§3.1) to prioritize masked locations for deriving reference versions. After that, Infilling-based Reference Version Generator (§3.2) infills the selected masked locations to derive reference versions and failing tests.

3.1 Context-aware Mask Location Prioritization (CamPri)

This subsection describes the workflow of CamPri, which can be divided into the following five stages.

Stage 1: Parse a PUT into an AST. CamPri's main objective is to identify appropriate masked locations (AST nodes) for MITGEN to perform infilling, in order to derive reference versions.

To identify appropriate masked locations, CamPri first parses a PUT into an AST, and considers each AST node of type Statement as a possible masked location, following the assumption made by CAPGEN [106] that a PUT's fault is located at one of its Statement nodes.

Stage 1 of Figure 2 shows an example of this stage. CamPri first parses the PUT into an AST. Then, for each AST node, CamPri replaces the code snippet represented by the AST node with a <MASK> token. We call the resultant

Algorithm 1: CamPri’s algorithm.

```

input :PUT, PUT’s requirement,  $K$ 
output:  $K$  prioritized masked locations, test inputs

// Stage 1: Parse a PUT into an AST
1 ast  $\leftarrow$  Parse PUT into an AST;
2 contextSimMap  $\leftarrow$  Initialize a Map to store the context similarity CamPri computes for each AST node in Stage 4;
3 for astNode  $\leftarrow$  ast do
    // Stage 2: Infill each masked location
    maskedPUT  $\leftarrow$  Mask the PUT by replacing the snippet represented by astNode with a <MASK> token;
    infilledPUTArray  $\leftarrow$  Initialize an array to store maskedPUT that is going to be infilled by snippets generated by LLM;
    for  $i \leftarrow 1$  to 10 do
        infilledPUT  $\leftarrow$  Generate a snippet and infill it into maskedPUT;
        infilledPUTArray[i]  $\leftarrow$  infilledPUT;
    if testInputs has not been generated then
        // Stage 3: Generate test inputs
        seedInputs  $\leftarrow$  generate seed inputs using LLM by providing the LLM the PUT and its requirement;
        testInputs  $\leftarrow$  generate additional inputs by mutating seedInputs;
    // Stage 4: Compute context similarity between generated snippets and original snippet
    // Check whether at least one infilledPUT is semantically different from the PUT
    outputInconsistent  $\leftarrow$  False;
    for  $i \leftarrow 1$  to 10 do
        for input  $\leftarrow$  testInputs do
            infilledPUT  $\leftarrow$  infilledPUTArray[i];
            outputInconsistent  $\leftarrow$  return True if PUT and infilledPUT have inconsistent output with respect to input;
            if outputInconsistent then
                break;
        if outputInconsistent then
            break;
    // Compute context similarity when at least one infilledPUT is semantically different from the PUT
    if outputInconsistent then
        GeneOriginal, VarOriginal, DepOriginal  $\leftarrow$  Compute the three contexts for original PUT;
        contextSimArray  $\leftarrow$  Initialize an array to store the context similarity CamPri computes for each pair of infilled PUT and original PUT;
        for  $i \leftarrow 1$  to 10 do
            infilledPUT  $\leftarrow$  infilledPUTArray[i];
            GeneInfill, VarInfill, DepInfill  $\leftarrow$  Compute the three contexts for infilledPUT;
            contextSimilarity  $\leftarrow$  Compute overall context similarity between (GeneInfill, GeneOriginal), (VarInfill, VarOriginal) and (DepInfill, DepOriginal);
            contextSimArray[i]  $\leftarrow$  contextSimilarity;
        avgContextSimilarity  $\leftarrow$  Take average of all context similarity values in contextSimArray;
        contextSimMap[astNode]  $\leftarrow$  avgContextSimilarity;
    else
        continue;

// Stage 5: Prioritize masked location
33 Sort contextSimMap in descending order of the avgContextSimilarity value it stores;
34 prioritizedMaskLocations  $\leftarrow$  Get top  $K$  astNode from the sorted contextSimMap;
35 return prioritizedMaskLocations, testInputs

```

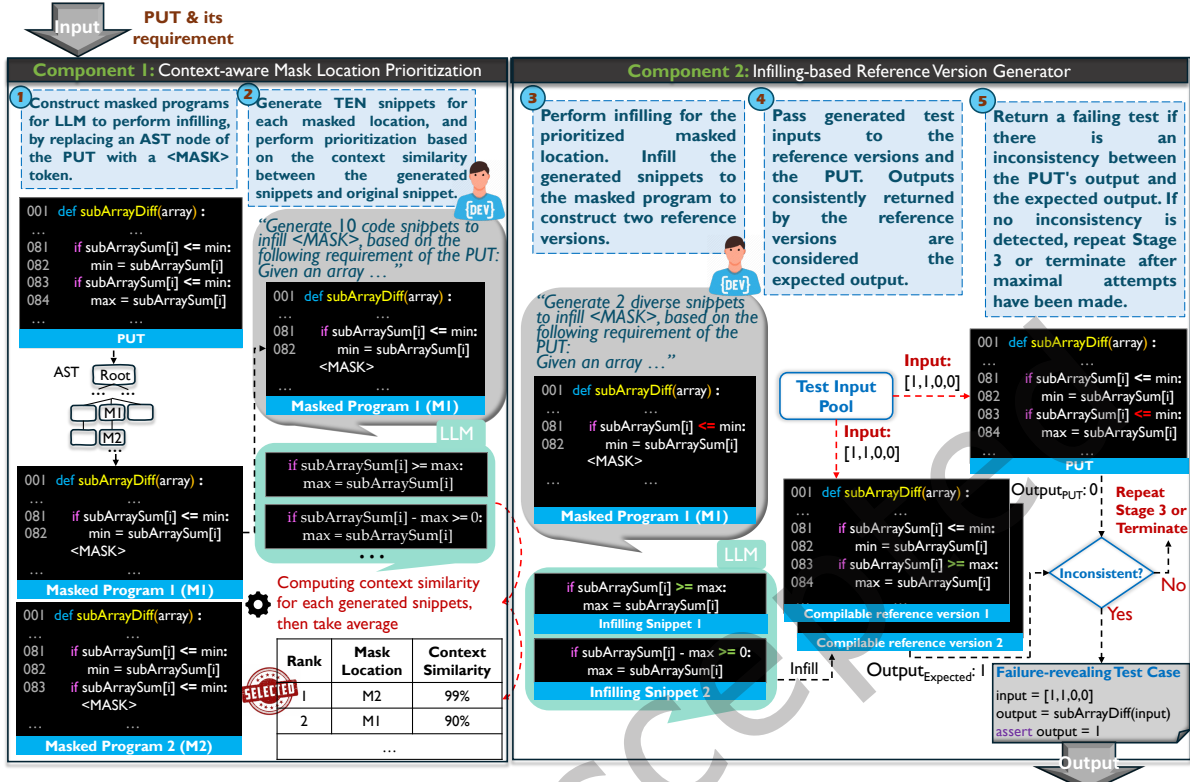


Fig. 2. Workflow of MitGEN.

program “Masked Program” (Stage 1 of Figure 2 shows two examples). These masked programs essentially specify the masked locations for LLM to perform infilling.

Stage 2: Infill each masked location. CamPri performs infilling for each masked location, in order to compute the context similarity between the generated snippets and the original snippet. As LLM is stochastic, to mitigate the randomness in the process of infilling, CamPri follows the setting of related works [28, 55] to generate ten snippets for each masked location, using temperature 0.8. Existing works [28, 55] consider ten snippets to be sufficient in mitigating randomness in code generation.

Stage 2 of Figure 2 shows an example of this stage. For each masked program generated in Stage 1, CamPri prompts an LLM to generate ten snippets for each masked program. Then, CamPri infills the ten snippets to the corresponding masked program, resulting in ten infilled programs corresponding to each masked program.

Stage 3: Generate test inputs. Test inputs are required by the following stages of CamPri and IRVGen. Following Liu et al. [55], CamPri generates test inputs by mutating a set of seed inputs generated by LLMs based on the PUT and its requirements. CamPri terminates the test input generation process when the branch coverage of PUT reaches 100% or saturates for two minutes, following an existing practice [33].

Stage 4: Compute context similarity between generated snippets and original snippet.

CamPri passes test inputs generated in Stage 3 and the ten infilled programs derived in Stage 2. If all the ten infilled programs have consistent outputs with the original program for all test inputs, CamPri excludes this masked location from prioritization. It is because this result indicates that snippets generated by the LLM for this

masked location have a high probability of being semantically equivalent to the original snippet. That is, snippets generated for this masked location is unlikely to reveal faults.

For masked locations not excluded by CamPri, CamPri computes the context similarity between the generated snippets and the original snippet. Specifically, CamPri first computes the context similarity between each generated snippet and the original snippet, following the equations described in §2.2. Then, CamPri calculates the average of context similarities CamPri computed between each generated snippet and original snippet.

Stage 5: Prioritize masked location. CamPri ranks masked locations according to the context similarity CamPri computes for each masked location in Stage 4. Mask locations that have a greater context similarity value are ranked first: a greater context similarity value implies the generated snippets are aligned with the correct intention of original snippet, according to CAPGEN’s finding [106]. Eventually, CamPri selects the top K masked locations. K is set to five by default. Our ablation study in §4.2.2.2 shows that MITGEN achieves the optimal F1-score when K is set to five.

Note that the snippets represented by a prioritized masked location may not be always faulty. In this case, MRTGEN’s IRVGen component is unlikely to return a failing test, according to Differential Prompting’s experiment result [52] and our evaluation result (4.1). We describe the workflow of IRVGen in §3.2.

Algorithm 2: IRVGen’s algorithm.

```

input : PUT, requirement, prioritizedMaskLocations, testInputs
output: a failing test or a message “failing test cannot be generated”

1 for astNode  $\leftarrow$  prioritizedMaskLocations do
    // Stage 1: Generate reference versions
2   maskedPUT  $\leftarrow$  Mask the PUT by replacing the snippet in astNode with a <MASK> token;
3   attempt  $\leftarrow$  0;
4   for attempt < 10 do
5     rv1, rv2  $\leftarrow$  infill maskedPUT;
6     if rv1 and rv2 have different syntax then
7       // Stage 2: Infer an expected output
8       for input  $\leftarrow$  testInputs do
9         outputrv1, outputrv2  $\leftarrow$  passes input to rv1 and rv2;
10        if outputrv1 == outputrv2 then
11          outputExpected  $\leftarrow$  outputrv1;
12          outputPUT  $\leftarrow$  pass input to PUT;
13          // Stage 3: Compare actual output and expected output
14          if outputExpected  $\neq$  outputPUT then
15            failTest  $\leftarrow$  construct a test case using input and outputExpected;
16            return failTest;
17          continue;
18        continue;
19   attempt += 1;
20 return “failing test cannot be found.”

```

3.2 Infilling-based Reference Version Generator

Infilling-based Reference Version Generator (IRVGen) accepts the top- K masked locations prioritized by CamPri, in order to derive reference versions and failing tests. IRVGen is divided into the following three stages.

Stage 1: Generate reference versions. IRVGen starts with the top-1 masked location ranked by CamPri. IRVGen generates two snippets for the masked location in order to construct two reference versions, following Differential Prompting’s test generation stage [52]. Stage 3 of Figure 2 illustrates the process of generating two snippets.

Then, IRVGen checks whether the two snippets have diverse syntax (i.e., differ by at least one AST node). It is because IRVGen follows Differential Prompting’s assumption: snippets generated by an LLM that have diverse syntax are unlikely to suffer from the same fault(s). In other words, for snippets generated by an LLM that have diverse syntax and have consistent output with respect to the same input, Differential Prompting’s assumption implies that the consistent output returned by these snippets is likely a correct output.

Hence, if the AST edit distance between two generated snippets is greater than zero, IRVGen accepts both snippets in order to derive reference versions and failing tests in the next Stage (Stage 2).

Otherwise, if IRVGen fails to generate snippets that have diverse syntax, or is unable to derive failing tests with the generated snippets in Stage 2, IRVGen regenerates the snippets. IRVGen makes at most ten attempts, following Differential Prompting’s setting. After ten attempts, if IRVGen is still unable to generate two snippets that have diverse syntax, or no failing test can be derived with the generated snippets in Stage 2, IRVGen stops further generating snippets for this masked location. IRVGen then proceeds to the next prioritized masked location and repeats this stage. MITGen terminates after making ten attempts on the fifth prioritized masked location and is still unable to generate a failing test.

Stage 2: Infer an expected output. If IRVGen successfully generates two snippets that have diverse syntax, IRVGen derives two reference versions from these two snippets, by infilling the two snippets into the corresponding masked program (Stage 4 of Figure 2 shows an example). Then, for each generated test input, IRVGen passes the input to these two reference versions. IRVGen checks whether the outputs returned by these two reference versions are consistent or not. If the outputs are consistent, IRVGen considers the consistent output an expected (correct) output of the PUT with respect to the test input.

If the two reference versions do not have a consistent output with respect to any generated test input, we consider that at least one of the reference versions is faulty. Hence, IRVGen regenerates two snippets for the same masked location (i.e., repeats Stage 1 of IRVGen). IRVGen makes at most 10 attempts, as discussed in Stage 1.

Stage 3: Compare actual output and expected output. In this stage, IRVGen examines whether the actual output of the PUT and the consistent output of the reference versions are inconsistent. If yes, the inconsistency between these two outputs essentially reveals a failure of the PUT. Hence, IRVGen constructs a failing test with the consistent output of the reference versions and the corresponding test input. IRVGen then returns this failing test to developers, as illustrated in Stage 5 of Figure 2.

If the two outputs are consistent, it implies that no failure has been revealed. Hence, IRVGen repeats Stage 2 with another test input. After IRVGen tries all test inputs and is still unable to reveal an inconsistency between the output of the PUT and the consistent output of the reference versions, IRVGen considers failing tests cannot be derived with the current reference versions. IRVGen then repeats Stage 1 to generate new snippets to derive new reference versions. IRVGen stops further finding failing tests after making the maximum number of attempts in deriving failing tests (as discussed in Stage 1).

3.3 Application Scenario

As discussed in §3, MitGen relies on program requirements of PUT to derive reference versions. One challenge of applying MitGen to real-world programs is that program requirements of arbitrary code may not always be available.

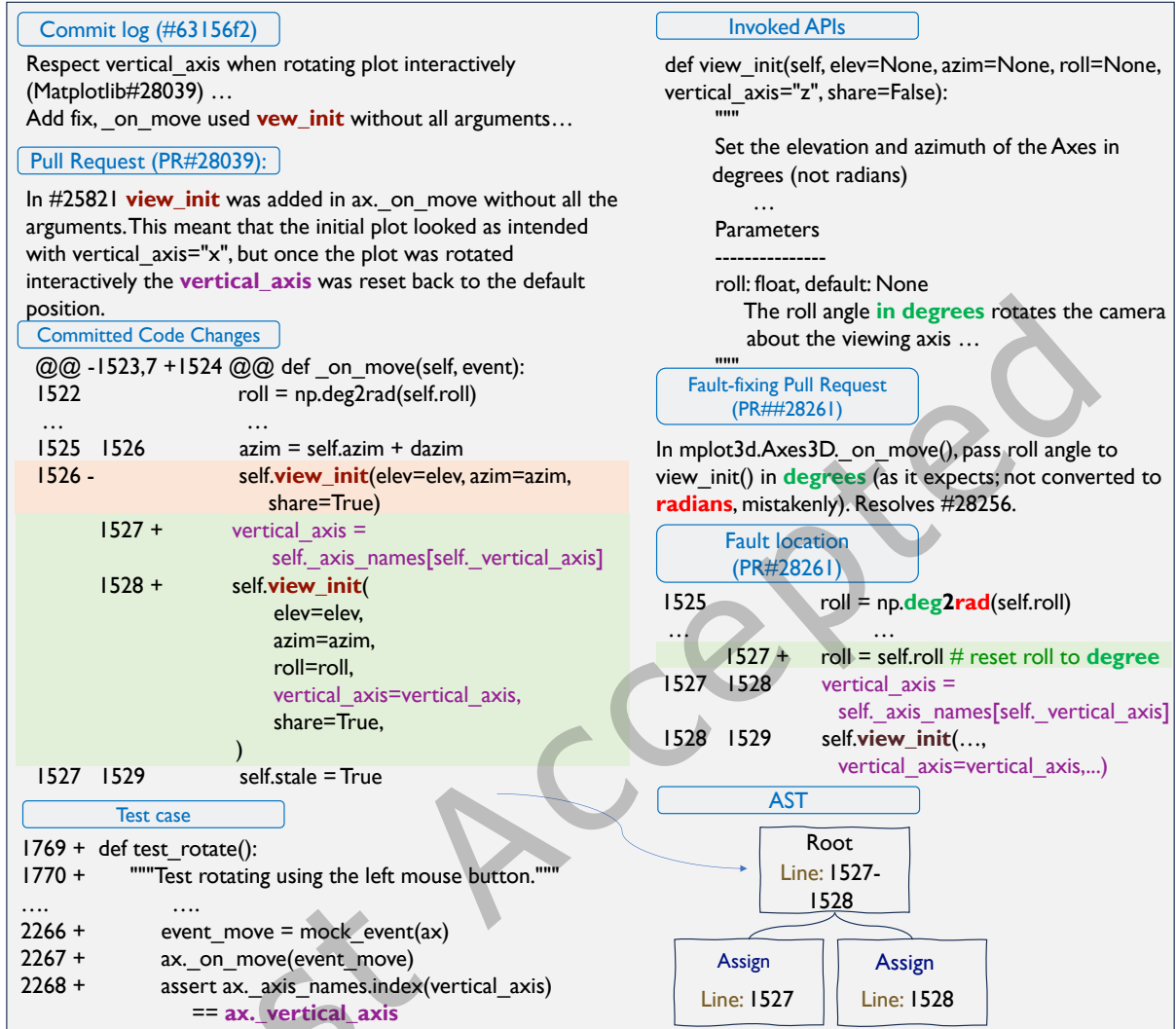


Fig. 3. An example of faulty code commits that MITGEN can be applied to generate failing tests.

To address this challenge, our insight is that **program requirements can be constructed from context information in code commits**. Specifically, when developers update project code through code commits, they are expected to include a description about *what* (which part of the code has been modified) and *why* (the intention of the modified code) [39, 40, 112]. An existing study [94] reveals that 84.2% of code commits that link to issue reports/pull requests describe both *what* and *why*. Furthermore, developers often bundle passing tests in code commits to validate the functionalities of modified code. These passing tests essentially serve as input-output pairs that can illustrate the functionalities of modified code. Both the intention of code and input-output pairs are essential pieces of contextual information required by LLM to generate code effectively [29, 49]. Based on

this insight, MITGEN exploits these two pieces of contextual information available in code commits to construct the program requirements.

Figure 3 shows an example of faulty code commit (#63156f2[66]) from matplotlib, which MitGen can be applied to reveal its fault. Specifically, the objective of this faulty code commit is to modify the arguments passed to `view_init()`. However, the developer made a fault that the `roll` variable was passed to `view_init()` as a variable in degree, while the `roll` variable should be in radian rather than in degree (as stated in the docstring of `view_init()`, also shown in Figure 3). To fix this fault, the `roll` variable should be converted back to radian before being passed to `view_init()`, as shown in line 1527 of the fault-fixing pull request #28261 [65].

MitGen can be applied to reveal this fault. Specifically, given the code commit #63156f2, MitGen exploits the context information (intention of modified code and input-output pairs) in this code commit to generate reference versions, and thus failing tests. Note that we limit MitGen’s application scenario to generate failing tests for only the snippets that have been modified (i.e., added/deleted/replaced code hunks), rather than snippets that subsume the modified snippets (e.g., function or class that contains the modified snippets). It is because the *what* and *why* information in a code commit is closely relevant to the modified snippets [94], and these pieces of information may not be closely relevant to the snippets other than the modified snippets. We leave the extension of MITGEN to more types of code snippets to future work.

Prompt Template: To derive reference versions, we propose a prompt template that exploits the contextual information in code commits. An example of the prompt template is shown in Figure 4, which comprises three sections: instructions, masked function, and contextual information:

- **Instructions** instructs LLM to locate the modified snippets and infill the `<MASK>` token in the modified snippets.
- **Masked function** shows the function which contains the `<MASK>` token to be infilled, where the modified snippets are annotated by “-” or “+”, and the `<MASK>` token is applied to one AST node of the modified snippets. Specifically, as discussed in an earlier paragraph of this section, MITGEN’s current application scenario is to generate failing tests for only the snippets modified in a code commit. Hence, the `<MASK>` token is applied to only the modified snippets. Besides, as discussed in §3.1, MITGEN assumes the fault located at only one AST node of type Statement, following the assumption of CAPGEN [106]. Hence, the `<MASK>` token is applied to only one AST node of type Statement in the modified snippets. An example of an AST derived from the modified snippets is shown in Figure 3, which comprises three nodes: a root node that represents line 1527-1528, two assignment nodes that represent line 1527 and 1528 respectively. Figure 4 shows three masked functions corresponding to each of these three AST nodes being masked. If multiple functions are modified in the modified snippets, only the masked function will be shown in this section, the remaining functions will be shown in the “*Relevant project code*” component in the third section as supplementary context. This is to avoid this section from being extra long, as extra long context may make LLM unable to attend to relevant tokens [105] (e.g., LLM may pay little attention to middle tokens if the tokens go extra long [57]).
- **Context information** comprises three main components: *Program requirements*, documentation of *Relevant APIs* (considered important by the state-of-the-art repository-level infilling approach [17]) and *Relevant project code* (considered important by [17]). The program requirements are composed of commit messages, pull request/issue messages and test cases bundled in the code commits. For API documentation (e.g., signature or docstring), MITGEN considers APIs invoked in modified snippets as relevant, because they are considered relevant by developers. MITGEN then collects the function signature and docstring of these APIs (if exists). The third component *Relevant project code* shows the code in the project files that contains the modified snippets. Annotations of the modified snippets and the `<MASK>` token are

# Instructions A developer has committed code changes for a project, but part of the code changes have been replaced with a <MASK> token. The lines annotated by "-" symbol indicate they were removed by the developer in this code commit. The lines annotated by "+" symbol indicate they were added by the developer in this code commit. Your task is to infill the <MASK> token with the given context. Output only the snippet for infilling <MASK>. If the context suggests no code is missing, output "No code is missing".		
# Code (Masked line 1527-1528) 1487 def _on_move(self, event): ... 1522 roll = np.deg2rad(self.roll) ... 1525 elev = self.elev + delev 1526 azim = self.azim + dazim - self.view_init(elev=elev, azim=azim, share=True) 1527-1528 + <MASK> 1529 self.stale = True ... 1555 self.figure.canvas.draw_idle()	# Code (Masked line 1528) 1487 def _on_move(self, event): ... 1522 roll = np.deg2rad(self.roll) ... 1525 elev = self.elev + delev 1526 azim = self.azim + dazim 1526 azim = self.azim + dazim - self.view_init(elev=elev, azim=azim, share=True) 1527 + vertical_axis = self._axis_names[...] 1528 + <MASK> 1529 self.stale = True ... 1555 self.figure.canvas.draw_idle()	# Code (Masked line 1527) 1487 def _on_move(self, event): ... 1522 roll = np.deg2rad(self.roll) ... 1525 elev = self.elev + delev 1526 azim = self.azim + dazim - self.view_init(elev=elev, azim=azim, share=True) 1527 + <MASK> 1528 + self.view_init(... vertical_axis=vertical_axis, ...) 1529 self.stale = True ... 1555 self.figure.canvas.draw_idle()
# Context information ## Commit log "Respect vertical_axis when rotating plot interactively (matplotlib#28039)..." ## Pull request "In #25821 view_init was added in ax._on_move without all the arguments..." ## Test case def test_rotate() ... ## Relevant APIs def view_init(self, elev=None, azim=None, roll=None, vertical_axis="z") ... ## Relevant project code <code>matplotlib/lib/mpl_toolkits/mplot3d/axes3d.py</code> 1 import collections ... 1526 azim = self.azim + dazim - self.view_init(elev=elev, azim=azim, share=True) 1527-1528 + <MASK> ... 3708 return x,y,z		
1 import collections ... 1526 azim = self.azim + dazim - self.view_init(elev=elev, azim=azim, share=True) 1527 + vertical_axis = self._axis_names[...] 1528 + <MASK> ... 3708 return x,y,z	1 import collections ... 1526 azim = self.azim + dazim - self.view_init(elev=elev, azim=azim, share=True) 1527 + <MASK> 1528 + self.view_init(... vertical_axis=vertical_axis, ...) ... 3708 return x,y,z	1 import collections ... 1526 azim = self.azim + dazim - self.view_init(elev=elev, azim=azim, share=True) 1527 + <MASK> 1528 + self.view_init(... vertical_axis=vertical_axis, ...) ... 3708 return x,y,z

Fig. 4. An example of prompt template adopted by MITGEN to derive reference versions.

preserved to ensure the context provided by the *Masked function* section and this section is consistent. Figure 4 shows an example of the prompt.

Model		
Name	Knowledge cutoff-date	
Starcoder2-15B-instruct-v0.1	6 September, 2023 [9]	
Meta-Llama-3-8B-Instruct	March, 2023 [68]	
CodeQwen1.5-7B-Chat	30 December, 2023 [78]	
Dataset		
Name	Release date	#Subjects
ClassEvalV1.0.0	3 January, 2024 [30]	100
EvoEvalV0.1.0	28 March, 2024 [109]	100
Real-world Subjects	After 1 January, 2024	100

Table 1. Models and Datasets used in our evaluation.

4 Evaluation

To evaluate MITGEN’s effectiveness and usefulness, we study the following three research questions. For each research question, we first outline the experiment design (e.g., subject selection, evaluation metrics, and baseline construction), followed by experimental results and discussions.

- 🟡 **RQ1 (Generating failing tests):** *Can MITGEN effectively generate correct failing tests?*
- 🟡 **RQ2 (Ablation Study)** *How effective are MITGEN’s components, i.e., IRVGen and CamPri?*

Environment. Our experiments were conducted on five computers: (1) Intel(R) Core(TM) i9-10980XE CPU @ 3.00GHz, 250GB RAM, two NVIDIA GeForce RTX 2080 Ti and one NVIDIA TITAN RTX, (2) Intel(R) Xeon(R) CPU E5-2683 v4 @ 2.10GHz, 251GB RAM, seven NVIDIA GeForce RTX 2080 Ti, (3) AMD Ryzen Threadripper PRO 3975WX 32-Cores, 503GB RAM, four NVIDIA GeForce RTX 3090, (4) AMD Ryzen Threadripper PRO 3995WX 64-Cores, 503GB RAM, three NVIDIA GeForce RTX 4090, (5) AMD EPYC 7773X 64-Core Processor, 1007 GB, two NVIDIA RTX 6000 Ada. Our replication package includes detailed documentation on how assignments were allocated to these devices.

4.1 RQ1: Can MITGEN effectively generate correct failing tests?

4.1.1 Experiment Setup.

Model. We conduct experiments using three popular LLMs: StarCoder2-15B-instruct-v0.1 [60], Meta-Llama-3-8B-Instruct [68] and CodeQwen1.5-7B-Chat [41]. These models are included in LLM leaderboards (e.g., Evalplus [55], Bigcodebench-leaderboard [124]), and are evaluated by recent works on code/test generation [45, 72, 102], showing that these models are representative of real-world LLMs on coding tasks related to MITGEN.

Dataset. Our evaluation is conducted on EvoEval-Difficult [107] and ClassEval [23]. We chose these datasets because, first, both benchmarks comprise challenging (non-trivial) coding tasks [23, 107]. Second, these benchmarks are released after the knowledge cut-off date of the selected models. Hence, the threat of data leakage can be mitigated. Table 1 shows the knowledge cut-off date of each model and the release date of each benchmark.

EvoEval-Difficult is a code generation benchmark that implements 100 function-level code generation tasks. Each coding task is adapted from coding tasks of HumanEval [10]. Specifically, HumanEval is a popular code generation benchmark released in 2021 [22, 23, 107]. Each subject implements a coding task commonly performed by developers (e.g., sorting, numerical calculation, string manipulation) [107]. As HumanEval is created before the knowledge cutoff date of recent LLMs, evaluation on it may result in the data contamination threat [22, 23, 107]. In addition, HumanEval’s subjects implement coding tasks widely exist on public portals, so evaluation on HumanEval may not effectively reveal an LLM’s performance on real-world coding tasks [107]. EvoEval-Difficult is proposed to address these two limitations. Specifically, the authors selected 100 coding tasks

from HumanEval, and adapted these coding tasks' requirements to make the coding tasks more challenging. For instance, the authors added less-common constraints or constraints that require more reasoning steps than the original requirements.

ClassEval [23] is a code generation benchmark for class-level code generation. It consists of 100 subjects. Compared to EvoEval-Difficult and HumanEval, ClassEval aims at addressing the limitation of earlier benchmarks (e.g., HumanEval [10]), that these benchmarks comprise only single-function code generation tasks, oversimplifying real-world coding tasks. For instance, single-function code generation tasks do not require LLM to consider dependencies between functions, invocation of libraries, and so on. Hence, ClassEval constructs class-level code generation tasks that incorporate essential features of real-world software, such as interdependent methods, invocation of libraries [23].

Real-world subjects comprise 100 faulty subjects selected from two sources: (1) SWE-Bench-Live [122], and (2) 164 projects covered in SWE-Bench-Live. We choose SWE-Bench-Live because, to the best of our knowledge, it is currently the only issue-resolving benchmark that consists of subjects created after the knowledge cut-off dates of LLMs selected in our evaluation (i.e., 1st January 2024). Overall, SWE-Bench-Live provides 33 subjects that match our subject selection criteria (described in Step 1 below). We further collect 184 subjects from the 164 projects covered in SWE-Bench-Live that match our subject selection criteria but are not included in SWE-Bench-Live. Then, we randomly select 67 subjects from these 184 subjects, so that the number of real-world subjects in our evaluation is 100 (33+67), aligning with the number of subjects in EvoEval-Difficult and ClassEval (shown in Table 1).

- **Step 1:** We start with SWE-Bench-Live [122], which comprises 1,565 subjects and covers 164 real-world projects as of 19 Aug, 2025. Each subject is essentially a pull request that addresses an issue submitted to the repository. From the 1,565 subjects, we select subjects that fulfill the following criteria: (1) the pull request was created after the knowledge cut-off date of the selected LLMs (1st January 2024), in order to mitigate the threat of data leakage, (2) the pull request fixes a fault rather than addresses non-fault-related issues (e.g., introducing new features), (3) the fault located at only one AST node of type Statement, in order to meet the assumption of MITGEN and CAPGEN [106] (the assumption is discussed in Stage 1 of §3.1). (4) The pull request contains a bug-reproduction test, which is needed for evaluating the effectiveness of reference versions. After this filtering, we obtain 229 subjects that satisfy these criteria.
- **Step 2:** In this step, for each fault-fixing pull request collected in Step 1, we identify the commit that introduces the fault. The fault-inducing version serves as the subject for experiments. Specifically, we adopt the state-of-the-art SZZ technique LLM4SZZ [90] to identify fault-inducing commits. For the fault-introducing commits returned by the LLM4SZZ, we assigned two authors to independently validate the returned commits. Discrepancies are resolved through modulations in the meetings with other authors. Finally, for the validated fault-introducing commits, we filter subjects based on the following two criteria: (1) there is only one fault-introducing commits corresponds to the fault, as MitGen's current application scenario is to reveal faults from the modified code of a single code commit (discussed in §3.3), (2) the fault-introducing commit includes passing tests for validating the modified code. The passing tests serve as input-output examples that are essential pieces of information for LLM to generate reference versions [29, 49], (3) the fault-introducing commit links to a pull-request or an issue, as pull request or issue describes the intention of modified code [39, 40, 112]. After this subject selection process, we obtain 33 subjects from SWE-Bench-Live.
- **Step 3:** To obtain up to 100 subjects, for the remaining 67 subjects, we further collect new pull requests from the GitHub repositories of the 164 projects covered in SWE-Bench-Live. We manually collect new subjects rather than apply SWE-Bench-Live's automatic subject collection method. Specifically, SWE-Bench-Live develops an LLM agent RepoLaunch to automatically collect issue-linking pull requests from

repositories. However, the authors of SWE-Bench-Live have not open-sourced RepoLaunch. Hence, we adapt the manual approach adopted by SWE-Bench [46], a benchmark prior to SWE-Bench-Live that also provides issue-resolving pull requests. Specifically, from each of the 164 project repositories on Github, we collect pull requests that satisfy three criteria: 1) the pull request has been merged, 2) the pull request resolves a GitHub issue, and 3) the pull request makes changes to the test files, which indicates that developers likely contributed tests to examine whether the issue has been resolved. We further add two additional criteria: 4) the pull requests address a fault (e.g., contain keywords “fix”, “bug”, “fault” in its title or content), 5) the pull request was created after the knowledge cut-off date of the selected LLMs in our evaluation (i.e., 1st January, 2024). After scanning 5233 subjects from the 164 projects, we obtain 184 subjects in total. Then, we sample 67 subjects from these 184 subjects. For each sampled subject, we further check whether they satisfy the subject selection criteria described in Step 1 and Step 2. If a subject fails to satisfy the subject selection criteria, we discard this subject and sample another subject from the 184 subjects, until we obtain 67 subjects that satisfy all these criteria.

During this subject collection process, we manually validate a subject’s fulfillment of certain subject selection criteria, such as (1) and (2) in step 2. To reduce manual effort in the validation processes, existing LLM-based approaches [6, 121] can be adapted. For instance, SWE-rebench [6] fine-tunes an LLM to assess the quality (i.e., Issue Clarity, Task Complexity or Test Patch Correctness) of issue-linking pull requests. The fine-tuned model can be applied to verify (2) of step 2. REPOLAUNCH [121] is an agentic framework that automatically validates the execution environment and patch correctness in pull requests. It can be used to validate (1) of step 2 by verifying whether the fault-introducing commit can actually be addressed by the fault-fixing commit.

These real-world subjects essentially emulate end-to-end scenarios of generating failing tests for code commits with MITGEN. Specifically, as discussed in §3.3, MITGEN generates failing tests for code commits by operating on several pieces of contextual information available in the code commit (i.e., textual information in commit log and pull request/issue report) and project code. Hence, in the end-to-end application scenario of MITGEN, the inputs of MITGEN include 1) a code commit, 2) pull request/issue report that links to the code commit, 3) the project’s source code whose version matches the code commit. Every real-world subject in our dataset provides this complete set of inputs, which mirrors the information a developer would have when working with an arbitrary GitHub repository. Indeed, an existing work [73] also operates on the same set of inputs for code commit validation in real-life scenarios, indicating our real-world subjects align with established practice for validating code commits.

Regarding the scale of these real-world subjects (committed code changes), they modify on average 148.1 lines/2.8 methods of the project code (excluding modifications to test files), indicating code changes in code commits are usually small in scale (also observed in an earlier work [3]). Hence, MITGEN is expected to be scalable on these subjects.

Bug injection. EvoEval-Difficult and ClassEval are both code generation benchmarks rather than benchmarks for debugging/repairing. They do not contain faulty code subjects for assessing an LLM’s performance in generating failing tests. To construct faulty subjects for EvoEval-Difficult, we adopt faulty subjects that an earlier work [69] created for HumanEval. As EvoEval-Difficult subjects are adapted from HumanEval subjects, each EvoEval-Difficult subject corresponds to one HumanEval subject. Hence, two authors independently migrate bugs from faulty HumanEval subjects to corresponding EvoEval-Difficult subjects, following an open coding scheme.

For ClassEval, unlike EvoEval-Difficult, no faulty subject has been created for ClassEval. Hence, we apply an existing bug injection tool for Python code (named FIT4Python [63]). FIT4Python can seed 16 categories of bugs (such as incorrect boolean expression, incorrect value for variable assignment). In each execution of FIT4Python, it first randomly selects one category of bugs to be seeded. Then, it randomly selects a snippet to

seed selected bugs. For each subject of ClassEval, we apply FIT4Python to randomly generate one faulty code for each ClassEval subject.

For Real-world Subjects, as real-world faults are available, we do not inject faults for these subjects.

Baselines. We compare MITGEN with three baselines, Differential Prompting, BaseLLM and Pynguin [61].

- Differential Prompting [52] is the state-of-the-art approach in generating failing tests.
- BaseLLM is a strawman approach that directly prompts an LLM to generate failing tests. This baseline is also included in Differential Prompting [52]’s evaluation. Specifically, this prompts an LLM in two steps. First, it checks with an LLM if a PUT contains bugs. Upon an affirmative response, it further prompts an LLM to generate a failing test. The two-step prompting convention does not assume any knowledge if a given program is faulty, emulating a common real-life situation [87].

We note that several LLM-based test generation works [24, 86, 119] have been proposed recently. These works inherently point out that providing additional context information (e.g., requirement of focal method) to LLM can enhance an LLM’s performance in test generation. Although these works are not proposed to find failing tests, it is possible that these works can also enhance an LLM’s performance in finding failing tests. Hence, we adopt the prompting strategy proposed by ChatTester [119], the state-of-the-art unit test generation approach, into the test generation step of BaseLLM. Specifically, ChatTester points out that providing LLMs with a PUT’s intention can enhance the LLM’s performance in generating test cases. We choose ChatTester because it requires only the availability of a PUT’s requirement, which is available in MITGEN’s application scenario or our evaluation datasets. Other related works require context information (e.g., a PUT’s usage example) that is not available in MITGEN’s application scenario or evaluation datasets.

- Pynguin [61] is the state-of-the-art unit test generation tool for Python.
- Fuzz4All [108] is a state-of-the-art LLM-based fuzzer. We select this approach because to the best of our knowledge, among all LLM-based fuzzers, only Fuzz4ALL targets general software and faults, other LLM-based fuzzing approaches target specific types of software, including Deep-learning libraries [18, 19], kernel [114], protocol [67], IoT [99]).

Comparison. We adopt the evaluation method presented in Differential Prompting’s paper [52] to evaluate MITGEN, Differential Prompting and BaseLLM. Specifically, for each evaluation subject, we execute each approach once. Hence, an approach returns either 0 or 1 test case. We then categorize these test cases to compute the evaluation metrics (more discussion about the evaluation metrics below). To mitigate randomness in an experiment, we repeat the experiment four times. Hence, for each subject, an approach generates 0 or four test case in total.

For Fuzz4All, we follow the experiment setup in its paper [108] which uses a fuzzing budget of 24 hours for each subject. We run this experiment on RTX 6000 Ada, which is computationally more powerful than the GPU used by Fuzz4All’s paper (i.e., RTX A6000) [108]. Hence, the time budget of 24 hours should be sufficient. We follow Fuzz4All’s paper [108] to program crash (implicit oracle) as the test oracle. Similar to the experiment setup of aforementioned approaches, we apply Fuzz4All four times for each subject. Each time Fuzz4All generates 0 or 1 failing test.

For Pynguin, as its main purpose is regression testing, it often generates more than one test case for a subject. Besides, the generated test cases’ main purpose is to achieve high code coverage, rather than to reveal faults. Hence, it would be unfair to adopt the mentioned method (i.e., requiring an approach to return only 0 or 1 test case) to evaluate Pynguin. For fair comparison, we use a different method to evaluate Pynguin. Specifically, for each subject, we run Pynguin for five minutes, following the setting in Pynguin’s paper [61]. Note that our servers are equipped with more and faster CPUs than the servers reported in Pynguin’s paper [61]. Thus, the time budget of five minutes should be sufficient. As the execution of Pynguin often returns more than one test

case, we assess the performance of Pynguin based on all test cases it generates in an execution (more discussion about the evaluation metrics below). We repeat this experiment four times, similar to the evaluation of MITGEN, Differential Prompting and BaseLLM.

Metrics. Following Differential Prompting’s paper [52], we divide generated failing tests into five categories. We use Figure 1 as an example to illustrate these categories.

- **Correct failing test (FT-IA).** An FT-IA consists of (1) a fault-triggering test input I that triggers a PUT to give an incorrect output, and (2) a correct assertion A that identifies the incorrect output. So, both the fault-triggering test input and assertion are correctly generated. An example is a test case that executes `subArrayDiff([1, 1, 0, 0])` and asserts its expected output to be 1. It correctly reveals a fault in `subArrayDiff`.
- **Coincidental failing test (FT-Ia):** An FT-Ia is similar to an FT-IA except its assertion a specifies an incorrect expected output. A test failure is coincidentally observed because the incorrect output returned by the PUT happens to be different from the incorrect output specified in the assertion. It consists of (1) a fault-triggering test input (I) that triggers a CUT’s fault, and (2) an assertion (a) that specifies an incorrect output. An example of FT-Ia is a test case that executes `subArrayDiff([1, 1, 0, 0])` but incorrectly asserts its expected output to be a value (e.g., 10) other than 1 (correct output) or 0 (an incorrect output of the PUT). An FT-Ia does not help fault diagnosis and program repair. It rejects correct patches.
- **False failing test (FT-ia):** An FT-ia reports a false alarm. The test case consists of (1) a non-fault-triggering test input i , and (2) an incorrect assertion a that asserts a wrong expected output. Unlike FT-Ia, the fault-triggering input of FT-ia is incorrectly generated in addition to incorrect assertion. An example is a test case that executes `subArrayDiff([1, 1, 1, 1])` and incorrectly asserts its expected output to be 3.
- **Passing test (PT)** A PT is a passing test.
- **Illegal argument test case (IT).** An IT consists of an illegal test input violating the designated argument type.

Out of the five categories, only an FT-IA is considered **correct failing test**, because it reveals a true failure. FT-Ia, FT-ia and IT are considered **incorrect failing tests**, because they reveal false failures. PT-ia and PT-Ia are not considered failing tests, because they do not reveal a failures.

For MITGEN, Differential Prompting and BaseLLM, as discussed in earlier paragraphs, each execution of them returns 0 or 1 test case. For faulty subjects, each execution can be divided into four categories: True Positive (TP), False Positive (FP), True Negative (TN), and False Negative (FN). An execution is considered a TP if it returns a correct failing test. An execution is considered an FP if it returns an incorrect failing test. An execution is considered a TN if it does not return failing test for correct PUT. An execution is considered an FN if it does not return failing test for faulty PUT.

To evaluate the performance of MITGEN, Differential Prompting and BaseLLM in finding failing tests for faulty subjects, we compute *precision* and *recall* based on the number of TP, FP, FN computed for all faulty subjects. Precision is calculated as $\#TP / (\#TP + \#FP)$, recall is calculated as $\#TP / (\#TP + \#FN)$. For correct subjects, as an execution cannot be categorized into TP or FN, but only FP or TN, we compute only the *false positive rate*, which is calculated as $\#FP / (\#FP + \#TN)$. As we repeat an experiment four times, we compute the overall *precision* and *recall*, and *false positive rate* of an approach, by averaging the results of these metrics.

For Pynguin, as it is a coverage-guided test generation approach rather than a failing test generation approach, Pynguin generates more than one test case in each execution. To fairly assess Pynguin’s effectiveness in finding failing test, in each execution on a faulty subject, we check whether there is at least one failing test among all generated test cases. If so, we consider the execution of Pynguin a TP. We then compute recall for Pynguin using the same equation as that used by MITGEN, Differential Prompting and BaseLLM (i.e., $\#TP / (\#TP + \#FN)$).

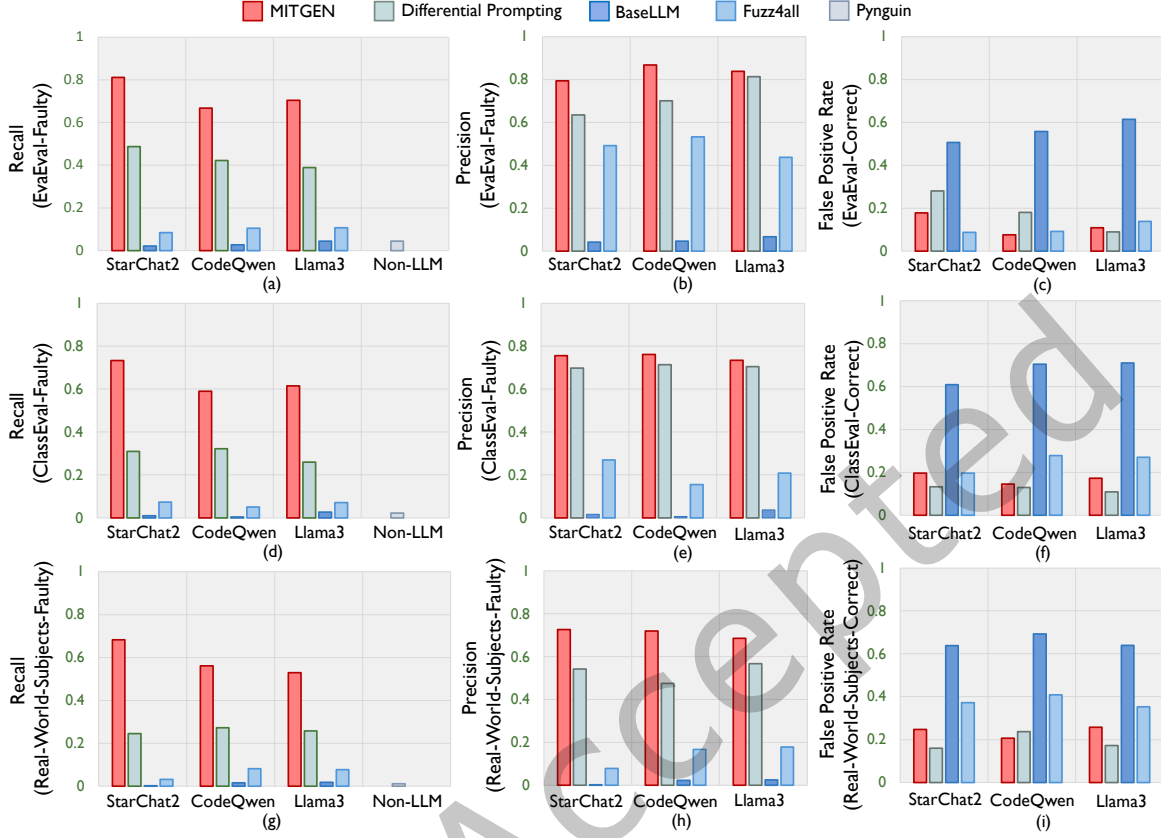


Fig. 5. Effectiveness of MITGEN and baselines in generating failing tests.

Validation of failing tests: We follow the methodology of existing works [47, 48] on fault-reproducing test generation to validate the correctness of failing tests. Specifically, we consider a failing test successfully reveals the fault of faulty code if it fails on the faulty code and passes on the correct code.

Note that we do not manually validate these failing tests. It is because to avoid subjectivity bias, existing works on test generation [54, 98, 103, 110] often request project developers to confirm that the generated tests truly reproduce the faults residing in the project. As we have generated thousands of failing tests, requesting manual inspection from developers for these failing tests may overwhelm the developers. Hence, we essentially rely on existing practice [47, 48] of validating the correctness of failing tests.

4.1.2 Experimental Results. Figure 5 shows MITGEN and baselines' performance in finding correct failing tests. For faulty subjects of EvoEval-Difficult and ClassEval, MITGEN's overall recall is 0.66, 112.7% improvement over the best baseline (0.31). Besides, MITGEN's precision is 0.77, comparable to the best baseline (0.65).

MITGEN outperforms Differential Prompting on both precision and recall. This is because MITGEN derives reference versions from a PUT's constituent snippets, rather than from an entire PUT. Table 2 shows that MITGEN's precision and recall in generating reference versions notably outperform Differential Prompting. We further analyze MITGEN's effectiveness in generating reference versions in §4.2.1. BaseLLM achieves

significantly worse performance than MITGEN, because LLMs are intrinsically weak at detecting faults [52]. Pynguin is a regression-based test generation approach, thus is ineffective in generating failing tests, except faults that can be detected by implicit oracles [35, 52].

Similarly, Fuzz4All achieves low recall because it adopts program crash as the test oracle. Hence, Fuzz4All can only detect limited types of faults. In addition, Fuzz4All achieves low precision because it often returns incorrect tests that have illegal program arguments. In contrast, MITGEN derives failing tests from the consistent outputs of reference versions, assuring that failing tests can be properly executed without illegal program arguments.

Figure 6 delves into the number of correct/incorrect failing tests generated by MITGEN or the baselines for each subject. Essentially, the average number of correct failing tests generated by MITGEN for each subject is 6.6, an 112.7% enhancement over Differential Prompting (3.1). We observed that the number of incorrect failing tests generated by MITGEN can be slightly greater than that of Differential Prompting. One main reason is that MITGEN generates more incorrect reference versions than Differential Prompting, as shown in Table 2. We further analyze the reasons and mitigation in §4.2.1.

Although MITGEN generates subtly more incorrect failing tests than Differential Prompting, we argue that the number of incorrect failing tests generated by MITGEN is acceptable. In fact, in a recent survey [118] on developers' expectation on test generation tools, 100% of practitioners will be satisfied with a test generation tool if over 80% of test cases generated by it are correct. As MITGEN's false positive on faulty subject is only 19.8%, and that on correct subject is only 18.2%, MITGEN meets this criteria. Besides, this survey points out that 92% practitioners will be satisfied with test generation tools if its bug detection rate is at least 60%. MITGEN (with 0.66 recall) also meets this criteria, while Differential Prompting (with 0.31 recall) is far from meeting this criteria.

Nevertheless, the false positive rate still indicates that when MITGEN is applied to many functions or code commits, the number of false positives can accumulate to a notable quantity. This condition could be exacerbated under the competent programmer hypothesis [2] that posits most functions or commits are correct. We consider the non-negligible false positive rate of MITGEN as a major limitation of MITGEN. In RQ2.1 (§4.2.1.2), we discuss the mitigation strategies.

Figure 5 further shows that MitGen's recall on real-world subjects is slightly lower than its recall on EvoEval and ClassEval. It is because coding tasks in repository-level subjects often entail longer and broader cross-file/in-file context [12, 50, 58]. Precisely identifying relevant context could be a challenging task for LLM [12, 50]. Nevertheless, compared to the baselines, the decrease in MitGen's performance is subtler. It is because MitGen decomposes an entire (complicated) coding task into smaller (simpler) coding tasks, reducing the amount of context information need to be captured by LLM to generate intended reference versions. In other words, MitGen reduces the difficulty for LLM to generate intended reference versions. In RQ2.1, we further analyze a few cases which MitGen achieves 100% recall with all the selected LLM, and the subjects that MitGen achieves 0% recall.

Failure Analysis. We analyze the few cases where MITGEN fails to find failing tests successfully and observe two situations. The first occurs when MITGEN is unable to find a failure-inducing test input, which often happens when the fault resides within a loop. Since MITGEN relies on branch coverage to assess test adequacy, it may struggle to generate failure-revealing inputs that expose such faults [8]. One possible solution is to incorporate loop coverage into the coverage metric. This would require developers to select a subset of loop paths to be covered, in order to balance testing effectiveness and efficiency [100]. The second case is that MITGEN is unable to generate reference versions that allows correct failing tests to be derived. We discuss this problem more in-depth in §4.2.1.

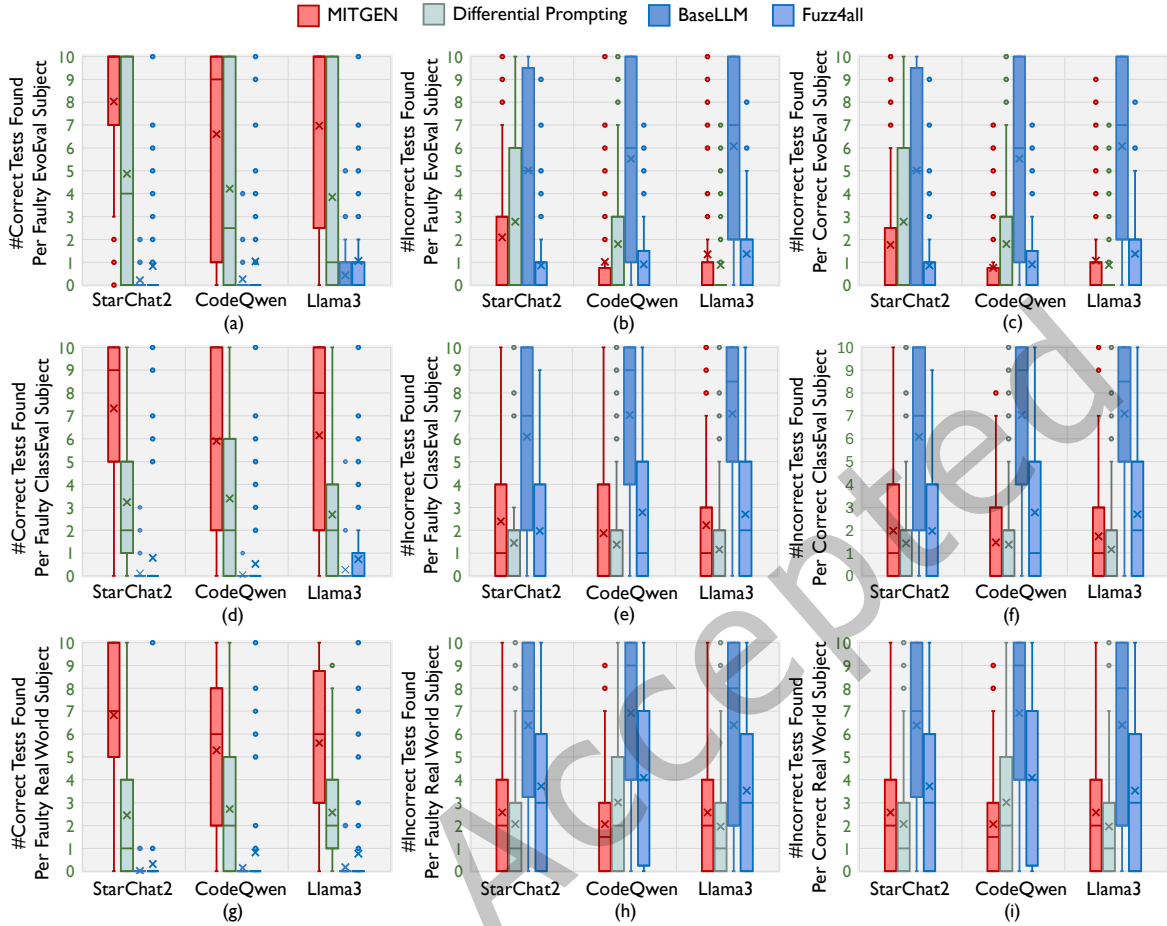


Fig. 6. Number of failing tests generated by MITGEN and the baselines.

Finding 1: MITGEN's overall recall is 0.66, 112.7% as the best baseline (0.31). MITGEN's overall precision is 0.77, favorably comparable to the best baseline (0.65).

Implication: MITGEN can effectively find failing tests, and obtains significant improvement in recall over Differential Prompting.

4.2 RQ2: How effective are MitGen's components, i.e., IRVGen and CamPri?

4.2.1 RQ2.1: Can MITGEN effectively generate reference versions with IRVGen?

4.2.1.1 Experiment Setup. To evaluate MITGEN's effectiveness in generating reference versions, we investigate whether MITGEN can generate reference versions that meet the two criteria described in §2.1.

Baseline. MITGEN's baseline is Differential Prompting (described in §2.1). MITGEN essentially extends Differential Prompting's reference version generation stage. The main difference is that MITGEN derives

Table 2. Effectiveness in generating reference versions.

Metrics	<i>StarCoder2-15B-instruct-v0.1</i>			<i>CodeQwen1.5-7B-Chat</i>			<i>Meta-Llama-3-8B-Instruct</i>		
	MITGEN	Differential Prompting	Improvement	MITGEN	Differential Prompting	Improvement	MITGEN	Differential Prompting	Improvement
<i>EvoEval</i>									
<i>Recall</i>	0.85	0.56	51.79% ↑	0.66	0.43	53.49 % ↑	0.71	0.39	82.05% ↑
<i>Precision</i>	0.84	0.62	35.48% ↑	0.85	0.64	32.81 % ↑	0.83	0.73	13.70% ↑
<i>False Positive</i>	0.23	0.47	51.06% ↑	0.12	0.22	45.45% ↑	0.15	0.14	-7.14%
<i>ClassEval</i>									
<i>Recall</i>	0.77	0.29	148.28% ↑	0.59	0.38	55.26% ↑	0.64	0.25	156.00% ↑
<i>Precision</i>	0.70	0.61	14.75% ↑	0.71	0.64	10.94% ↑	0.72	0.52	38.46% ↑
<i>False Positive</i>	0.24	0.19	-26.32%	0.24	0.21	-14.29%	0.25	0.22	-13.64%
<i>Real world subjects</i>									
<i>Recall</i>	0.67	0.23	198.89% ↑	0.59	0.29	103.48% ↑	0.62	0.23	170.33% ↑
<i>Precision</i>	0.70	0.43	61.54% ↑	0.71	0.49	45.34% ↑	0.71	0.42	70.61% ↑
<i>False Positive</i>	0.29	0.28	-4.46%	0.25	0.3	15.83% ↑	0.25	0.32	21.09% ↑

reference versions from selected snippets of a PUT, while Differential Prompting derives reference versions from an entire PUT.

Metrics. MITGEN or Differential Prompting generates two reference versions for each subject. Similar to RQ1, we classify each pair of two reference versions into True Positive (TP), False Positive (FP), True Negative (TN) and False Negative (FN). The definitions are as follows:

- **True Positive (TP):** a pair is considered TP if (1) the PUT is faulty, (2) the two reference versions return a correct and consistent output for at least one failure-revealing input. That is, at least one correct failing test can be derived from the two reference versions.
- **False Positive (FP):** a pair is considered FP if (1) the reference versions do not return a correct and consistent output for at least one failure-revealing input (if exists), (2) the reference versions return a consistent but incorrect output for at least one test input. That is, only incorrect failing test can be derived from these reference versions.
- **False Negative (FN):** a pair is considered FN if (1) the PUT is faulty, (2) the reference versions do not have any consistent output, or the consistent output returned by the reference versions for any test input is always the same as that returned by the faulty PUT. That is, no failing test is derived from the reference versions for the faulty PUT.
- **True Negative (TN):** a pair is considered TN if (1) the PUT is correct, (2) the reference versions do not have any consistent output, or the consistent output returned by the reference versions for any test input is always the same as that returned by the correct PUT. That is, no failing test is derived from the reference versions for the correct PUT.

Validation of reference versions: Technically, for each pair of reference versions that MITGEN generated for a PUT, we exercise developers' test suite bundled in the PUT to assess the reference versions' effectiveness. Based on the test result, we classify the pair of these two reference versions into TP, FP, TN or FN based on the test results:

- **True Positive (TP):** the pair is classified as TP if 1) the PUT is faulty, 2) at least one of the fault-reproducing tests in the bundled test suite is passed by both reference versions. The satisfaction of these two conditions indicates that at least one correct failing test can be derived from the reference versions.
- **False Positive (FP):** the pair is classified as FP if all consistent outputs of both reference versions are different from that of the correct version and the faulty version (if exists) with respect to the same test inputs. In other words, only incorrect failing tests can be derived.

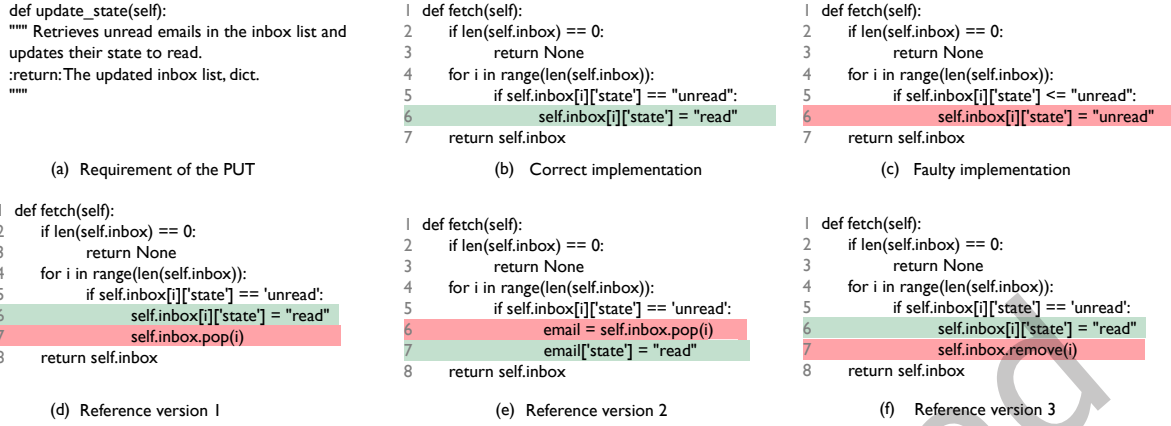


Fig. 7. An example illustrating the failure of LLMs in generating reference versions. This example is adapted from an evaluation subject CLASSEVAL_EMAILCLIENT. This example implements a function in a class that handles email services. Specifically, this function updates the state of inboxed emails from “unread” to “read”. Figure (a) shows the requirement of this method. Figure (b) shows the correct implementation of this method. Figure (c) shows the faulty implementation, where the fault is located at line 6: the correct line should be `if self.inbox[i]['state'] == "read":` while the faulty implementation is `if self.inbox[i]['state'] == "unread":`. Suppose reference versions are generated by infilling line 6-7, which cover the fault location. Figure (d), (e) and (f) show three examples of the reference versions typically generated by the evaluated LLM. These reference versions fixed the original fault, yet they inherently introduced an extra statement `self.inbox.pop(i)`, which removes the element of key `i` from the dictionary `self.inbox`. The extra statement causes these reference versions to return an incorrect output. These figures show examples of LLM’s misunderstanding of ambiguous program requirements (see §4.2.1).

- **False Negative (FN):** the pair is classified as FN if (1) the PUT is faulty, and (2) no failing test has been derived from these two reference versions.
- **True Negative (TN):** the pair is classified as TN if (1) the PUT is correct, and (2) no failing test has been derived from these two reference versions.

We further perform manual inspection on the reference versions that are classified as TP, in order to check whether they actually fix the faults. Following an open coding procedure [15], we assign two authors to independently inspect whether the TP reference versions actually fix the fault. Each authors follow the labeling procedure adopted in earlier works [56, 115]: reference versions satisfying the following criteria are considered successful in fixing the faults: (1) the fault-fixing snippet in reference versions is syntactically identical to the developer patches, or (2) the fault-fixing snippet in reference versions are semantically equivalent to the developer patches according to the rules summarized proposed in [115]. Discrepancies between the two authors are discussed until a consensus is reached.

As more than a thousand TP reference versions have been generated, we sample 111 reference versions for manual inspection, following the sample size proposed by an existing work on program repair [14]. Our manual inspection result shows that all sampled TP reference versions successfully fix the fault, indicating that developers’ test suite can effectively identify TP reference versions.

Similar to RQ1, we compute *precision* and *recall* based on the number of TP, FP, FN computed for all the faulty subjects. Precision is computed as $\#TP/(\#TP+\#FP)$, recall is computed as $\#TP/(\#TP+\#FN)$. For correct subjects, as

a pair of reference versions cannot be categorized into TP or FN, but only FP or TN, we compute only the *false positive rate*, calculated as $\#FP/(\#FP+\#TN)$.

Comparison. Recall that in RQ1, MITGEN or Differential Prompting is applied to each faulty/correct subjects once in each experiment. We categorize the executions of an approach into TP, FP, and FN, based on the definitions mentioned in earlier paragraphs. Then we compute *precision*, *recall* and *false positive rate*. As the experiment is run four times, we compute the overall *precision* and *recall*, and *false positive rate* of an approach, by averaging the result of these metrics computed in each of the four experiments.

4.2.1.2 Experimental Result. Table 2 shows MITGEN’s average recall in generating reference versions across all models and datasets is 0.65, 103.1% over Differential Prompting (0.32). MITGEN’s precision in generating reference versions is 0.75, 38.9% over the Differential Prompting. MITGEN achieves higher recall because CamPri and IRVGen allow MITGEN to effectively derive reference versions from a PUT’s code snippets, rather than from an entire program. By doing so, MITGEN can overcome an LLM’s limitation in generating code for non-trivial coding tasks.

We further compute the Pearson correlation between MITGEN’s recall in generating good reference versions and its recall in generating failing tests. We obtain a correlation value 0.82, indicating a very high correlation [34, 70]. The main reason is that MITGEN derives failing tests via differential testing between reference versions and the original PUT, thus MITGEN’s success rate in generating good reference versions is imperative to its success rate in generating failing tests.

This table also shows that MITGEN has a higher recall on EvoEval-Difficult than ClassEval and Real-world Subjects. One possible reason is that EvoEval-Difficult implements single-function coding tasks, while ClassEval implements class-level coding tasks. In fact, earlier studies [24, 120] point out that LLMs are weak at generating class-level code that comprises interdependent functions/variables. Fortunately, recent studies [12, 117, 120] point out that in-context learning techniques can enhance an LLM’s capability in generating class-level code [12].

100% Recall Case Analysis. To further understand MITGEN’s performance, we look into a couple of subjects that all the three models achieve 100% recall. We found that these subjects essentially adapt algorithms commonly seen on public portals (e.g., Github). For instance, evo-58 adapts a classical algorithm “longest substring”. classeval-blackjackgame implements a popular card game *blackjack*.

Another example can be found in pull request#915 [76] of python-babel/babel. This pull request implements a common function that statically extracts the value of string variables in string literals. However, the code change provided in this pull request does not handle the case that the string literal contains line breaks [75]. Selected LLMs successfully incorporate this case in the reference versions, possibly due to the commonality of this function.

A recent study [79] reveals that LLMs’ performance on a code generation task can be positively correlated with the exposure of a coding task’s implementation on GitHub. The finding reveals that greater exposure to a coding task on public portals can lead to a greater possibility that an LLM has been well-trained on the task. This finding provides a possible explanation to MITGEN’s performance on these tasks.

0% Recall Case Analysis. We further study 29 subjects for which all three models achieve 0% recall, or attain a false positive rate higher than 20% (20% is the maximum incorrect rate of test cases generally acceptable by developers [118]). Our analysis revealed that both cases mainly stem from ambiguous program requirements, leading the LLM to generate unintended code. While such ambiguity may not confuse human developers, it can mislead LLMs [89], likely due to their weaker reasoning capabilities and heightened sensitivity to natural language ambiguities [89]. For instance, Figure 7 shows a method implementing an email client, with the requirement to update the state of emails from “unread” to “read.” Figures 7(d), (e), and (f) display typical reference versions generated by the three LLMs evaluated. Although these versions successfully fix the original fault, they

inadvertently introduce an incorrect statement—`self.inbox.pop(i)`—which removes the entry with key `i` from `inbox`.

One possible reason is that the meaning of “Retrieve unread emails” in the requirement is ambiguous to LLM. The intended meaning should be “Identify unread emails”, but LLMs seemed to interpret this phrase as “Take emails away”. Hence, the LLMs include the incorrect statement (`self.inbox.pop(i)`) in the generated code. To fix this problem, we replace “Retrieve” with a less ambiguous term (e.g., “Identify”). After the replacement, the LLMs no longer include the incorrect statement in the generated code. We have a similar observation on the remaining 9 subjects, indicating that the clarity of requirements is essential to the effectiveness of LLMs in generating code.

Another example of an ambiguous description can be found in `theskumar/python-dotenv#552` [92], which both the commit message and the pull request only comprise one sentence “Some more s/Python-dotenv/python-dotenv/”. In reality, “s/Python-dotenv/python-dotenv/” means standardizing the naming convention from “Python-dotenv” to “python-dotenv” (all lowercase). However, without additional context, this expression appears to be ambiguous. By providing LLM a clearer expression, i.e., standardizing the naming convention from “Python-dotenv” to “python-dotenv” (all lowercase), all the LLM successfully generates the intended reference versions.

To address LLMs’ misunderstandings of program requirements in code generation tasks, one possible solution is to apply a recent approach μ FiX [95], a prompting technique that prompts LLM to review the test results of code it generates based on given program requirements, guiding the LLM to identify and refine its misunderstanding of the program requirements. We leave the integration of MITGEN and μ FiX to future work.

Interestingly, for subjects with ambiguous descriptions, we found that MITGEN is more likely to accept the incorrect reference versions for deriving failing tests than Differential Prompting. One possible reason is that MITGEN derives reference versions at snippet-level while Differential Prompting derives reference versions at program-level. Hence, LLMs often need to generate less tokens in MITGEN’s reference version generation process than that of Differential Prompting. An earlier study [32] observed a negative correlation between token length and semantical diversity of the tokens. This implies that reference versions generated by MITGEN are more likely to have similar semantic (consistent outputs) than those generated by Differential Prompting. Hence, for incorrect reference versions, MITGEN are more likely to accept them, causing incorrect failing tests to be derived. This reason also provides a possible explanation for MITGEN’s slightly higher false positive rate in generating reference versions, as shown in Table 2.

Another case is that the subjects implement a mathematical problem. We found that there are 3 subjects (`evo-39`, `evo-46`, `evo-49`) fall into this case. Recent studies [83] also show that LLM has weak mathematical capability, that it often achieves poor performance in solving mathematical problems. To address this issue, recent studies [4] show that supervised fine-tuning techniques can notably enhance LLMs’ mathematical capability.

 **Finding 2:** MITGEN’s recall in generating reference versions is 0.65, 103.1% over Differential Prompting (0.32). MITGEN’s precision in generating reference versions is 0.75, favorably comparable to Differential Prompting (0.54).

 **Implication:** MITGEN can effectively generate reference versions that allows correct failing tests to be derived.

4.2.2 RQ2.2: Can CamPri prioritize masked locations that allow reference versions to be effectively generated?

In this RQ, we evaluate CamPri’s effectiveness in prioritizing masked locations. Specifically, CamPri’s objective is to prioritize masked locations that have high likelihood to generate intended reference versions (i.e., reference versions that fulfill the two criteria described in §2.1). For the ease of presentation, we call reference versions that satisfy these two criteria **good reference versions**. We call masked locations that good reference versions have been generated **good mask locations**.

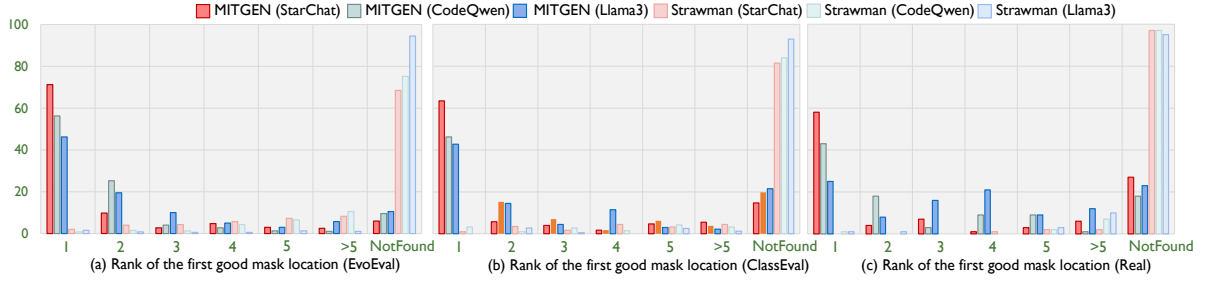


Fig. 8. Effectiveness of CamPri.

Table 3. CamPri's contribution to MITGEN's performance in failing test generation.

Metrics	StarCoder2-15B-instruct-v0.1			CodeQwen1.5-7B-Chat			Meta-Llama-3-8B-Instruct		
	MITGEN w/ CamPri	MITGEN w/o CamPri	Improvement	MITGEN w/ CamPri	MITGEN w/o CamPri	Improvement	MITGEN w/ CamPri	MITGEN w/o CamPri	Improvement
EvoEval									
Recall	0.85	0.14	507.14% ↑	0.66	0.08	725.00% ↑	0.71	0.1	610.00% ↑
Precision	0.84	0.26	223.08% ↑	0.85	0.22	286.36% ↑	0.83	0.32	159.38% ↑
False Positive	0.23	0.39	41.03% ↑	0.12	0.29	58.62% ↑	0.15	0.21	28.57% ↑
ClassEval									
Recall	0.72	0.07	928.57% ↑	0.59	0.05	1080.00% ↑	0.64	0.05	1180.00% ↑
Precision	0.7	0.14	400.00% ↑	0.71	0.1	610.00% ↑	0.72	0.11	554.55% ↑
False Positive	0.26	0.36	27.78% ↑	0.24	0.38	36.84% ↑	0.25	0.38	34.21% ↑
Real world subjects									
Recall	0.67	0.04	1581.25% ↑	0.59	0.02	2825.00% ↑	0.62	0.04	1437.50% ↑
Precision	0.70	0.14	401.68% ↑	0.71	0.09	690.27% ↑	0.71	0.10	608.93% ↑
False Positive	0.29	0.38	30.94% ↑	0.25	0.43	70.29% ↑	0.25	0.39	35.26% ↑

4.2.2.1 Experimental Setup. Baseline. To the best of our knowledge, there is no related work that prioritizes masked locations based on their likelihood of being good mask locations. Hence, we construct a strawman approach that directly prompts an LLM to prioritize masked locations. It prompts an LLM in two steps. First, it checks with an LLM if a PUT contains bugs. Upon an affirmative response, it further prompts an LLM to prioritize five AST nodes that allow good reference versions to be generated.

Metrics. To the best of our knowledge, no metric has been proposed to assess an approach's effectiveness in prioritizing masked locations based on their likelihood of being good mask locations. As CamPri is essentially inspired by CAPGEN's insight in prioritizing plausible patches, we consider CAPGEN the most relevant work of CamPri. Hence, we adapt the evaluation metrics proposed by CAPGEN. Specifically, CAPGEN uses *Rank of the first correct patch* to assess an approach's effectiveness in patch prioritization. We adapt this evaluation metric by computing the rank of the first masked location (among the top-five selected masked locations) that is a good mask locations. We call it **Rank of the first good mask location**. As CamPri selects only five masked locations in each execution, there are chances that none of the five selected masked locations is a good mask location. Hence, to compute the Rank of the first good mask location, we consider only the subjects whose five masked locations selected by CamPri include at least one good mask location. To assess CamPri's completeness, we also compute *recall*. Similar to RQ1 and RQ2.1, we categorize an execution of CamPri into True Positive (TP) or False Negative (FN). An execution is considered TP if the prioritized masked locations contain at least one good mask location. An execution is considered FN if the prioritized masked locations do not contain any good mask location. Then, recall is calculated as $\#TP / (\#TP + \#FN)$.

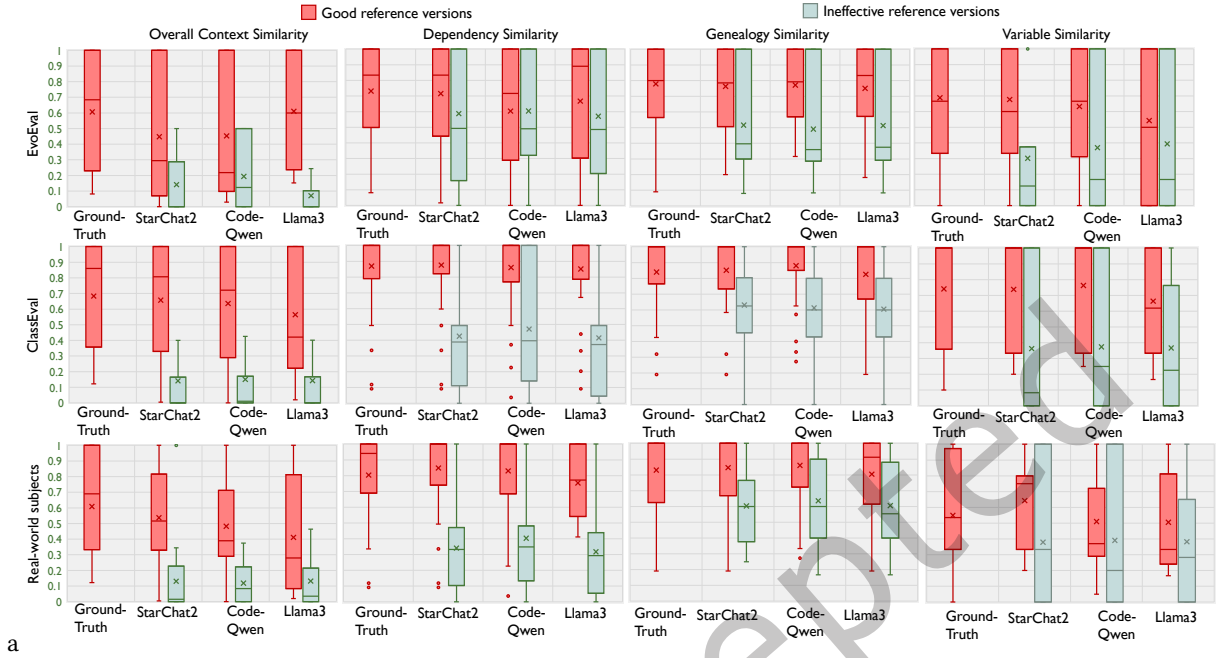


Fig. 9. Context similarity between good reference versions and other reference versions.

Comparison. Recall that in RQ1, in each execution of MITGEN, CamPri selects five masked locations that are likely good mask locations. We compute the *recall* and *Rank of the first good mask location* based on these selected masked locations. As the experiment is repeated four times, we further take an average of the recall and the rank computed in each experiment. For the strawman approach, we execute it on each faulty subject four times, same as MITGEN. As the strawman approach also selects five masked locations for each subject, we compute the recall and the rank using the same method as MITGEN.

To further study CamPri's contribution to MITGEN's performance in generating reference versions, we conduct an ablation study to investigate MITGEN's performance without CamPri. Recall that in RQ2.1, we study MITGEN's performance with CamPri enabled. To study MITGEN's performance with CamPri disabled, we assess MITGEN's performance in generating reference versions for all AST nodes of a subject, rather than only the AST nodes selected by CamPri. Specifically, for each subject, suppose that there are N AST nodes, we apply MITGEN to each AST node once. Hence, MITGEN is executed N times for a subject. We then categorize each of these N executions into TP, FP, TN, and FN, following the definitions described in RQ2.1. Then, we count the frequencies of TP, FP, TN, and FN returned by MITGEN for a subject. For each of the frequencies of TP, FP, TN, and FN, we divide the frequency by N , in order to align with the setting of MITGEN when CamPri is enabled (i.e., MITGEN with CamPri enabled is executed only once for a subject rather than N times, so the frequency of TP, FP, TN, and FN is at most one rather than N). Finally, we compute *recall*, *precision*, and *false positive rate*, using the method described in RQ2.1.

4.2.2.2 Experimental Results. Figure 8 shows Rank of the first good mask location found by CamPri and the strawman approach. CamPri's recall is 0.81, 8.10X as the strawman approach (0.10). Further, the *Rank of the first*

Table 4. MITGEN’s performance in generating failing tests with respect to different values of K (i.e., number of prioritized mask locations selected for generating reference versions).

<i>K</i>	1	5	10	15	20
<i>Recall</i>	0.50	0.66	0.67	0.68	0.68
<i>Precision</i>	0.81	0.77	0.71	0.66	0.59
<i>F1-score</i>	0.62	0.71	0.69	0.67	0.63

good mask location computed by CamPri is 1.71, a notable enhancement over the strawman approach (2.12). These results show that CamPri is effective in prioritizing good mask locations.

We look into the reason that CamPri is effective in prioritizing good mask locations, by comparing the context similarity computed from good reference versions and that computed from other reference versions that are not good reference version (CamPri does not consider reference versions that are semantically the same as original snippets, see Stage 4 in §3.1).

We further study how CamPri contributes to MITGEN’s effectiveness of generating reference versions. Table 3 shows that when CamPri is enabled, MITGEN’s recall, precision and false positive rate significantly improves. For instance, MITGEN’s recall with CamPri enabled is 1208.22% higher than that with CamPri disabled. One possible reason is that not all AST nodes (masked locations) implement a coding task conducive to good reference versions, as discussed in §2.1. Hence, generating reference versions for every possible masked location (i.e., MITGEN with CamPri disabled) achieves only 8.17% recall in generating good reference versions. In comparison, CamPri prioritizes a small subset of masked locations that are likely good mask locations, allowing MITGEN to attain a much higher recall.

Overall, we compute the correlation between MitGen’s recall in prioritizing mask locations and its recall in generating failing tests. We obtain a correlation value 0.58, indicating a moderate to high correlation [34, 70]. One main reason is that successful generation of failing tests requires MITGEN to rank at least one good mask location in top five positions. Hence, MITGEN’s success rate in prioritizing mask locations is pivotal to MITGEN’s success rate in generating failing tests.

We analyze a few cases where CamPri fails to include a good mask location in the five masked locations prioritized by it. The major case is that an LLM misunderstands the PUT’s requirement, as discussed in §4.2.1. Hence, the generated reference versions are semantically different from the original snippets (Figure 7 shows an example), thus having low context similarity with the original snippets. We discuss the strategy for mitigation in §4.2.1.

Context similarity metrics. We further evaluate the effectiveness of context similarity metrics used by CamPri to prioritize locations (in Stage 5 of CamPri, as described in §3.1). Specifically, we study the effectiveness of each context similarity metrics in distinguishing good reference versions (i.e., reference versions that satisfy C1 and C2 defined in §2.1) and ineffective reference versions (i.e., reference versions that do not satisfy C1 or C2). Note that only reference versions that reveal at least one failing tests (i.e., having different behaviors from original PUT) will be considered in this experiment, as reference versions that satisfy this condition will be considered by CamPri for prioritization (as discussed in Stage 4 of §3.1).

Based on the classification, we first compute the context similarity metrics between good reference versions and faulty code, as well as the context similarity metrics between ineffective reference versions and faulty code. Then, we compare the context similarities metrics computed from good reference versions and ineffective reference versions respectively.

Figure 9 shows that the context similarity metrics computed from good reference versions is notably (2.0X) higher than that computed from ineffective reference versions. This result suggests that context similarity metrics

Table 5. Pearson correlations between context similarity metrics with respect to reference versions and behavioral similarity between the reference versions and correct code.

Metrics	Overall	Dependency	Genealogy	Variable
<i>EvoEval</i>				
<i>StarChat2</i>	0.31	0.33	0.27	0.38
<i>CodeQwen</i>	0.32	0.29	0.35	0.36
<i>Llama3</i>	0.24	0.31	0.33	0.18
<i>ClassEval</i>				
<i>StarChat2</i>	0.37	0.45	0.31	0.42
<i>CodeQwen</i>	0.39	0.47	0.40	0.34
<i>Llama3</i>	0.30	0.42	0.31	0.25
<i>Real-world subjects</i>				
<i>StarChat2</i>	0.38	0.51	0.30	0.42
<i>CodeQwen</i>	0.42	0.44	0.27	0.41
<i>Llama3</i>	0.32	0.49	0.33	0.25
Average	0.34	0.41	0.32	0.33

is an effective indicator for distinguishing good reference versions and ineffective reference versions, facilitating CamPri to prioritize good mask locations. This implication is further supported by the experimental finding that the overall context similarities between correct snippets and faulty snippets are higher than those between other reference versions and the faulty snippets. This result indicates that correct patch often share similar context with buggy code, as observed by our earlier work [106].

We further look into the correlation between the context similarity metrics with respect to reference versions and behavioral similarity between the reference versions and correct code. It is because the main objective of the context similarity metrics is to infer the behavioral similarity between the reference versions and correct code. To compute the behavioral similarity, we compute the percentage of developers' tests passed by the reference versions. A higher percentage indicates a reference version is behaviorally more aligned with the correct code. Then, we compute the Pearson correlation between the context similarity metrics and the behavioral similarity.

Table 5 shows that the correlation is 0.34, showing a moderate to high correlation [34, 70] between context similarity metrics and behavioral similarity. Specifically, for ClassEval and real-world subjects, Dependency context shows the highest correlation with behavioral similarity. One possible reason is that subjects in these two datasets are larger in scale, and dependency context is the only context whose elements are gathered from an entire program rather than from current function (genealogy and variable context considers only code elements in current function). Hence, there are more many more possible combinations for dependency context, and require the reference version to be more semantically (behaviorally) similar to the correct code in order to achieve high similarity in dependency context.

Ablation study of K. Table 4 shows an ablation study of MITGEN's performance with respect to different values of K. Our experimental result shows that MITGEN's F1 score is the optimal when K equals five. Specifically, although higher K increases the recall, the increase is marginal. It is because a majority of good reference versions can already be generated from the first 1.71 prioritized mask locations (as shown in Figure 8). On the other hand, greater K value can decrease MITGEN's precision because it increases the possibility of including incorrect reference versions in the process of deriving failing tests.

 **Finding 3:** CamPri’s recall is 0.81, 8.10X as the strawman approach (0.10). Besides, the *Rank of the first good mask location* computed by CamPri is 1.71, a notable enhancement over the strawman approach (2.12).
 **Implication:** CamPri is effective in prioritizing good mask locations.

 **Finding 4:** The context similarity between good reference versions and original snippets is 2.0X higher than that between other reference versions and original snippets.
 **Implication:** Context similarity is an effective indicator for CamPri to identify good reference versions, and thus allows CamPri to effectively prioritize good mask locations.

4.3 Discussion

Verification of failing tests. The experimental results show that MITGEN is effective in generating failing tests. However, MITGEN may still return incorrect failing tests. Hence, developers may need to manually inspect whether the failing test returned by MITGEN reveals true fault. To ease the manual inspection process, we suggest two strategies to aid developers in manual inspection. The first strategy is to exercise the failing test on an earlier and correct version of the code. This strategy is applicable to failing tests that detect regression faults. Another strategy to validate if a test failure truly reveals a fault is to examine the corresponding suspicious commits identified using automated techniques based on the test failure [11, 90].

Availability of intention or input-output examples. As discussed in §3.3, MITGEN requires the intention of PUT and also input-output examples to be provided by developers. Although these pieces of contextual information are available in all of our evaluation subjects, it is possible that this contextual information is not available in some real-life scenarios. Fortunately, a couple of LLM-based approaches [80, 119] have been proposed for intention inference or test generation. For instance, SpecOver [80] is an LLM-based agent that infers code intention. ChatTester [119] is a prompting strategy for generating unit tests. These approaches can assist developers in producing the intention or input-output examples for the PUT.

4.4 Threat to Validity

Our evaluation is conducted using Starcoder2-15B-instruct-v0.1, CodeQwen1.5-7B-Chat, Meta-Llama-3-8B-Instruct. Although these models are popular models that are evaluated by existing leaderboards [55, 124] and existing works [45, 72, 102], they are not the latest models upon the submission of this paper. Hence, there would be a concern that our evaluation result on MITGEN may not be generalized to the latest models. Nonetheless, MITGEN aims at addressing a current limitation of LLMs, that LLMs have worse performance in generating test cases for non-trivial coding tasks, as evidenced in recent works [52, 119]. A few studies [45, 101] explain that it is because non-trivial coding tasks require more sophisticated reasoning, while transformers are intrinsically weak at reasoning [25, 77].

Although recent advances in reinforcement learning (e.g., Group Relative Policy Optimization [85]) show promising performance in enhancing LLM’s reasoning capability, recent studies and empirical findings show that LLMs trained with reinforcement learning still suffer from poor performance on non-trivial tasks. For instance, LiveCodeBench [43] shows that for coding contest problems created after DeepSeek-R1’s release date (20th Jan 2025), DeepSeek-R1 achieves only 36.7% pass@1 for medium-level problems, and 0.0% pass@1 for hard-level problems. Hence, the results and findings in our paper can still provide useful references for future large language models.

5 Related Work

5.1 Test oracle generation

Automated test oracle generation is an outstanding challenge. Conventional test oracle generation paradigms such as metamorphic testing, differential testing often have limited application scenarios [96].

Recent, numerous works [20, 42, 84, 91, 104, 119] have been proposed to leverage DL models (including LLMs) to infer an assertion. These works can be divided into two categories: *training-based approaches* and *inference-based approaches*. Training-based approaches [20, 42, 104] pre-train or fine-tune DL models to infer an assertion based on test prefix. For instance, TOGLL explores the effectiveness of fine-tuning LLM with different prompts [36]. These works are orthogonal to MITGEN as MITGEN do not require pre-training/fine-tuning of DL models; such process may require excessive computational costs, and is not necessarily affordable by developers [38].

Another line of works study how pre-trained DL models can be leveraged to generate assertions. ChatTester [119] is a unit test generation approach based on ChatGPT. Its insight is that providing an LLM a PUT's intention can enhance the LLM's performance in generating unit tests. We construct a baseline of MITGEN by adapting ChatTester [119] (§4.1.1). Schafer et al. [84] also point out that additional contextual information (e.g., usage example of a CUT) can further improve an LLM's performance. Dakhel et al. [16] leverages mutation testing to generate feedback information for an LLM to refine its test case. Tang et al. [91] and Siddiq et al. [86] conducted an empirical study on using LLMs to generate tests, analyzing their strength and limitations in generating oracle, and providing actionable advice (e.g., combining LLM with SBST approaches).

Differential Prompting [52] combines LLM and differential testing to generate failing tests. Details about Differential Prompting have been discussed in §2.1 and it is one of the baseline in our evaluation.

5.2 Failure-revealing test input generation

Many works target the problem of generating test inputs. We divide these works into *regression-based*, *learning-based* and *fuzzing-based*. Regression-based approaches (e.g., coverage-guided test input generation [26, 61]) generate test inputs to achieve high code coverage. For example, Pynguin [61] generates test input to achieve high branch coverage. Our evaluation includes some of these state-of-the-art approaches (e.g., Pynguin [61] and ChatTester [119]).

Learning-based techniques generate test inputs by leveraging neural networks (including LLMs). TestGen-LLM [5] and CoverUp [71] propose prompting strategies for LLMs to generate tests, in order to increase test coverage or cover missing corner cases. SymPrompt [81] extracts path constraints and prompts LLM to generate test cases that cover diverse paths. TELPA [113] combines static analysis and LLM to generate diverse tests to achieve high branch coverage. Jiang et al. [45] combine constraint-based approaches and LLM to generate test inputs. Liu et al. [55] combine input mutation strategies and LLM to generate diverse test inputs. We adopt the test generation strategy proposed by Liu et al. to generate test inputs for MITGEN (§3.1).

Fuzzing-based techniques [53, 62, 82, 108] generate test inputs to trigger specific types of failures (e.g., crash, security vulnerabilities) from a CUT. Typically, a fuzzing approach either requires a test oracle to be pre-defined by developers, or automatically derived from implicit oracle [62, 108]. In comparison, MITGEN is not restricted to detecting specific types of failure. Also, MITGEN does not require a test oracle to be pre-defined or limited to only implicit oracle. Hence, we consider these works orthogonal to MITGEN.

6 Conclusion

In this paper, we propose MITGEN, a new technique for generating failing tests using LLMs. MITGEN aims at addressing a fundamental limitation of LLM on non-trivial task. Specifically, non-trivial coding task often require an LLM to perform sophisticated reasoning, while LLMs are intrinsically weak at reasoning. To address this limitation, MITGEN's insight is to derive failing tests from a PUT's snippets, rather than from the entire PUT. This

allows an LLM to derive failing tests from simpler coding tasks, overcoming an LLM's limitation on non-trivial tasks. MITGEN also addresses the challenge of prioritizing code snippets that allows MITGEN to effectively derive failing tests. Our evaluation result shows that MITGEN significantly outperforms state-of-the-art baselines. We provide a reproduction package at <https://mitgent.github.io/>.

7 Acknowledgment

We would like to thank the anonymous reviewers for their insightful comments. This work is partially supported by the National Science Foundation of China (Grant No. 92582201), the Hong Kong SAR Research Grant Council/General Research Fund (Ref No. 16206524), and the Hong Kong Research Grant Council/Theme-based Research Scheme (Ref No. T41-517/25-N).

References

- [1] AGHAJANI, E., NAGY, C., VEGA-MÁRQUEZ, O. L., LINARES-VÁSQUEZ, M., MORENO, L., BAVOTA, G., AND LANZA, M. Software documentation issues unveiled. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE) (2019)*, IEEE, pp. 1199–1210.
- [2] AHMED, Z., SCHWASS, E., HERBOLD, S., TRAUTSCH, F., AND GRABOWSKI, J. A new perspective on the competent programmer hypothesis through the reproduction of real faults with repeated mutations. *Software Testing, Verification and Reliability* 34, 3 (2024), e1874.
- [3] ALALI, A., KAGDI, H., AND MALETIC, J. I. What's a typical commit? a characterization of open source software repositories. In *2008 16th IEEE international conference on program comprehension* (2008), IEEE, pp. 182–191.
- [4] ALONSO, N. I., ET AL. The mathematics of deepseek-r1: Theoretical foundations and comparative analysis. *The Mathematics of DeepSeek-R1: Theoretical Foundations and Comparative Analysis (January 22, 2025)* (2025).
- [5] ALSHAHAWAN, N., CHHEDA, J., FINOGENOVA, A., GOKKAYA, B., HARMAN, M., HARPER, I., MARGINEAN, A., SENGUPTA, S., AND WANG, E. Automated unit test improvement using large language models at meta. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering* (2024), pp. 185–196.
- [6] BADERTDINOV, I., GOLUBEV, A., NEKRASHEVICH, M., SHEVTSOV, A., KARASIK, S., ANDRIUSHCHENKO, A., TROFIMOVA, M., LITVINTSEVA, D., AND YANGEL, B. Swe-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. *arXiv preprint arXiv:2505.20411* (2025).
- [7] BAL, J., BAL, S., CHU, Y., CUI, Z., DANG, K., DENG, X., FAN, Y., GE, W., HAN, Y., HUANG, F., HUI, B., JI, L., LI, M., LIN, J., LIN, R., LIU, D., LIU, G., LU, C., LU, K., MA, J., MEN, R., REN, X., REN, X., TAN, C., TAN, S., TU, J., WANG, P., WANG, S., WANG, W., WU, S., XU, B., XU, J., YANG, A., YANG, H., YANG, J., YANG, S., YAO, Y., YU, B., YUAN, H., YUAN, Z., ZHANG, J., ZHANG, X., ZHANG, Y., ZHANG, Z., ZHOU, C., ZHOU, J., ZHOU, X., AND ZHU, T. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).
- [8] BALUDA, M., DENARO, G., AND PEZZÈ, M. Bidirectional symbolic analysis for effective branch testing. *IEEE Transactions on Software Engineering* 42, 5 (2015), 403–426.
- [9] BIGCODE. Bigcode/the-stack-v2 · datasets at hugging face.
- [10] CHEN, M., TWOREK, J., JUN, H., YUAN, Q., PINTO, H. P. D. O., KAPLAN, J., EDWARDS, H., BURDA, Y., JOSEPH, N., BROCKMAN, G., ET AL. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [11] CHEN, X., ZHU, H., CAO, J., WEN, M., AND CHEUNG, S.-C. Semic: Semantic-aware identification of bug-inducing commits. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 1363–1385.
- [12] CHEN, Y., WU, J., LING, X., LI, C., RUI, Z., LUO, T., AND WU, Y. When large language models confront repository-level automatic program repair: How well they done? In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings* (2024), pp. 459–471.
- [13] CINISELLI, M., COOPER, N., PASCARELLA, L., MASTROPAOLO, A., AGHAJANI, E., POSHYVANYK, D., DI PENTA, M., AND BAVOTA, G. An empirical study on the usage of transformer models for code completion. *IEEE Transactions on Software Engineering* 48, 12 (2021), 4818–4837.
- [14] CLAUDEYJ. Patch correctness/semantic equivalence rules.md at master · claudeyj/patch correctness.
- [15] CRESWELL, J. W. *Qualitative Inquiry and Research Design: Choosing Among Five Approaches (3rd Edition)*. SAGE Publications, Inc., 2013.
- [16] DAKHEL, A. M., NIKANJAM, A., MAJDINASAB, V., KHOMH, F., AND DESMARAIS, M. C. Effective test generation using pre-trained large language models and mutation testing. *Information and Software Technology* 171 (2024), 107468.
- [17] DENG, L., REN, X., NI, C., LIANG, M., LO, D., AND LIU, Z. Enhancing project-specific code completion by inferring internal api information. *IEEE Transactions on Software Engineering* (2025).
- [18] DENG, Y., XIA, C. S., PENG, H., YANG, C., AND ZHANG, L. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis* (2023), pp. 423–435.

- [19] DENG, Y., XIA, C. S., YANG, C., ZHANG, S. D., YANG, S., AND ZHANG, L. Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries. In *Proceedings of the 46th IEEE/ACM international conference on software engineering* (2024), pp. 1–13.
- [20] DINELLA, E., RYAN, G., MYTKOWICZ, T., AND LAHIRI, S. K. Toga: A neural method for test oracle generation. In *Proceedings of the 44th International Conference on Software Engineering* (2022), pp. 2130–2141.
- [21] DONG, J., SUN, J., ZHANG, W., DONG, J. S., AND HAO, D. Contested: Consistency-aided tested code generation with llm. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 596–617.
- [22] DONG, Y., JIANG, X., LIU, H., JIN, Z., GU, B., YANG, M., AND LI, G. Generalization or memorization: Data contamination and trustworthy evaluation for large language models. In *Findings of the Association for Computational Linguistics: ACL 2024* (Bangkok, Thailand, Aug. 2024), L.-W. Ku, A. Martins, and V. Srikumar, Eds., Association for Computational Linguistics, pp. 12039–12050.
- [23] DU, X., LIU, M., WANG, K., WANG, H., LIU, J., CHEN, Y., FENG, J., SHA, C., PENG, X., AND LOU, Y. Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation. *arXiv preprint arXiv:2308.01861* (2023).
- [24] DU, X., LIU, M., WANG, K., WANG, H., LIU, J., CHEN, Y., FENG, J., SHA, C., PENG, X., AND LOU, Y. Evaluating large language models in class-level code generation. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (2024), pp. 1–13.
- [25] FAN, Z., GAO, X., MIRCHEV, M., ROYCHOUDHURY, A., AND TAN, S. H. Automated repair of programs from large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)* (2023), IEEE, pp. 1469–1481.
- [26] FRASER, G., AND ARCURI, A. Evosuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering* (2011), pp. 416–419.
- [27] FRASER, G., STAATS, M., MCMINN, P., ARCURI, A., AND PADBERG, F. Does automated unit test generation really help software testers? a controlled empirical study. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 24, 4 (2015), 1–49.
- [28] FRIED, D., AGHAJANYAN, A., LIN, J., WANG, S., WALLACE, E., SHI, F., ZHONG, R., YIH, W.-T., ZETTLEMOYER, L., AND LEWIS, M. InCoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999* (2022).
- [29] FU, Y., LI, B., LI, L., ZHANG, W., AND XIE, T. The first prompt counts the most! an evaluation of large language models on iterative example-based code generation. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 1583–1606.
- [30] FUDANSELAB. Release release classeval v1.0.0 · fudanselab/classeval.
- [31] GUO, D., YANG, D., ZHANG, H., SONG, J., ZHANG, R., XU, R., ZHU, Q., MA, S., WANG, P., BI, X., ET AL. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [32] GUO, Y., SHANG, G., AND CLAVEL, C. Benchmarking linguistic diversity of large language models. *arXiv preprint arXiv:2412.10271* (2024).
- [33] HOLMES, J., AHMED, I., BRINDESCU, C., GOPINATH, R., ZHANG, H., AND GROCE, A. Using relative lines of code to guide automated test generation for python. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 29, 4 (2020), 1–38.
- [34] HOPKINS, W. A new view of statistics, Apr 2003.
- [35] HOSSAIN, S. B. Ensuring critical properties of test oracles for effective bug detection. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings* (2024), pp. 176–180.
- [36] HOSSAIN, S. B., AND DWYER, M. Togl: Correct and strong test oracle generation with llms. *arXiv preprint arXiv:2405.03786* (2024).
- [37] HUANG, B., LU, S., WAN, X., AND DUAN, N. Enhancing large language models in coding through multi-perspective self-consistency. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (2024), pp. 1429–1450.
- [38] HUANG, K., ZHANG, J., BAO, X., WANG, X., AND LIU, Y. Comprehensive fine-tuning large language models of code for automated program repair. *IEEE Transactions on Software Engineering* (2025).
- [39] HUANG, Y., TANG, Z., CHEN, X., YANG, C., ZHENG, Z., AND ZHOU, X. Commit messages generation based on core changes. *ACM Transactions on Software Engineering and Methodology* (2025).
- [40] HUANG, Z., HUANG, Y., CHEN, X., ZHOU, X., YANG, C., AND ZHENG, Z. An empirical study on learning-based techniques for explicit and implicit commit messages generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (2024), pp. 544–556.
- [41] HUI, B., YANG, J., CUI, Z., YANG, J., LIU, D., ZHANG, L., LIU, T., ZHANG, J., YU, B., LU, K., ET AL. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).
- [42] IBRAHIMZADA, A. R., VARLI, Y., TEKINOGLU, D., AND JABBARVAND, R. Perfect is the enemy of test oracle. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (2022), pp. 70–81.
- [43] JAIN, N., HAN, K., GU, A., LI, W.-D., YAN, F., ZHANG, T., WANG, S., SOLAR-LEZAMA, A., SEN, K., AND STOICA, I. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974* (2024).
- [44] JIANG, X., DONG, Y., WANG, L., FANG, Z., SHANG, Q., LI, G., JIN, Z., AND JIAO, W. Self-planning code generation with large language models. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–30.
- [45] JIANG, Z., WEN, M., CAO, J., SHI, X., AND JIN, H. Towards understanding the effectiveness of large language models on directed test input generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (2024), pp. 1408–1420.
- [46] JIMENEZ, C. E., YANG, J., WETTIG, A., YAO, S., PEI, K., PRESS, O., AND NARASIMHAN, K. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770* (2023).

- [47] KANG, S., YOON, J., ASKARBEEKYZY, N., AND YOO, S. Evaluating diverse large language models for automatic and general bug reproduction. *IEEE Transactions on Software Engineering* (2024).
- [48] KANG, S., YOON, J., AND YOO, S. Large language models are few-shot testers: Exploring llm-based general bug reproduction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)* (2023), IEEE, pp. 2312–2323.
- [49] LI, J., ZHAO, Y., LI, Y., LI, G., AND JIN, Z. Acecoder: An effective prompting technique specialized in code generation. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–26.
- [50] LI, J., ZHU, H., LIU, H., SHI, X., ZONG, H., DONG, Y., ZHANG, K., JIANG, S., JIN, Z., AND LI, G. aixcoder-7b-v2: Training llms to fully utilize the long context in repository-level code completion. *arXiv preprint arXiv:2503.15301* (2025).
- [51] LI, N., LI, Y., LIU, Y., SHI, L., WANG, K., AND WANG, H. Drowzee: Metamorphic testing for fact-conflicting hallucination detection in large language models. *Proceedings of the ACM on Programming Languages* 8, OOPSLA2 (2024), 1843–1872.
- [52] LI, T.-O., ZONG, W., WANG, Y., TIAN, H., WANG, Y., CHEUNG, S.-C., AND KRAMER, J. Nuances are the key: Unlocking chatgpt to find failure-inducing tests with differential prompting. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (2023), IEEE, pp. 14–26.
- [53] LIANG, J., JIANG, Y., WANG, M., JIAO, X., CHEN, Y., SONG, H., AND CHOO, K.-K. R. Deepfuzzer: Accelerated deep greybox fuzzing. *IEEE Transactions on Dependable and Secure Computing* 18, 6 (2019), 2675–2688.
- [54] LIU, J., LIN, J., RUFFY, F., TAN, C., LI, J., PANDA, A., AND ZHANG, L. Nnsmith: Generating diverse and valid test cases for deep learning compilers. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (2023), pp. 530–543.
- [55] LIU, J., XIA, C. S., WANG, Y., AND ZHANG, L. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in Neural Information Processing Systems* 36 (2023), 21558–21572.
- [56] LIU, K., WANG, S., KOYUNCU, A., KIM, K., BISSYANDÉ, T. F., KIM, D., WU, P., KLEIN, J., MAO, X., AND TRAON, Y. L. On the efficiency of test suite based program repair: A systematic assessment of 16 automated repair systems for java programs. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering* (2020), pp. 615–627.
- [57] LIU, N. F., LIN, K., HEWITT, J., PARANJAPPE, A., BEVILACQUA, M., PETRONI, F., AND LIANG, P. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173.
- [58] LIU, T., XU, C., AND MCAULEY, J. Repobench: Benchmarking repository-level code auto-completion systems. In *The Twelfth International Conference on Learning Representations*.
- [59] LOZHKOVA, A., LI, R., ALLAL, L. B., CASSANO, F., LAMY-POIRIER, J., TAZI, N., TANG, A., PYKHITAR, D., LIU, J., WEI, Y., ET AL. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173* (2024).
- [60] LOZHKOVA, A., LI, R., ALLAL, L. B., CASSANO, F., LAMY-POIRIER, J., TAZI, N., TANG, A., PYKHITAR, D., LIU, J., WEI, Y., ET AL. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173* (2024).
- [61] LUKASCZYK, S., AND FRASER, G. Pynguin: Automated unit test generation for python. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings* (2022), pp. 168–172.
- [62] MANÈS, V. J., HAN, H., HAN, C., CHA, S. K., EGELE, M., SCHWARTZ, E. J., AND WOO, M. The art, science, and engineering of fuzzing: A survey. *IEEE Transactions on Software Engineering* 47, 11 (2019), 2312–2331.
- [63] MARQUES, H., LARANJEIRO, N., AND BERNARDINO, J. Injecting software faults in python applications: The openstack case study. *Empirical Software Engineering* 27, 1 (2022), 20.
- [64] MARTINEZ, M., AND MONPERRUS, M. Mining software repair models for reasoning on the search space of automated program fixing. *Empirical Software Engineering* 20 (2015), 176–205.
- [65] MATPLOTLIB/MATPLOTLIB. Correct roll angle units, issue #28256. <https://github.com/matplotlib/matplotlib/pull/28261>.
- [66] MATPLOTLIB/MATPLOTLIB. Respect vertical_axis when rotating plot interactively. <https://github.com/matplotlib/matplotlib/pull/28039/files>.
- [67] MENG, R., MIRCHEV, M., BÖHME, M., AND ROYCHOUDHURY, A. Large language model guided protocol fuzzing. In *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS)* (2024), vol. 2024.
- [68] META-LLAMA. Meta-llama/llama3: The official meta llama 3 github site.
- [69] MUENNIGHOFF, N., LIU, Q., ZEBAZE, A., ZHENG, Q., HUI, B., ZHUO, T. Y., SINGH, S., TANG, X., VON WERRA, L., AND LONGPRE, S. Octopack: Instruction tuning code large language models. *arXiv preprint arXiv:2308.07124* (2023).
- [70] OLAGUE, H. M., ETZKORN, L. H., GHOLSTON, S., AND QUATTLEBAUM, S. Empirical validation of three software metrics suites to predict fault-proneness of object-oriented classes developed using highly iterative or agile software development processes. *IEEE Transactions on Software Engineering* 33, 6 (2007), 402–419.
- [71] PIZZORNO, J. A., AND BERGER, E. D. Coverup: Coverage-guided llm-based test generation. *arXiv preprint arXiv:2403.16218* (2024).
- [72] POURCEL, J., COLAS, C., MOLINARO, G., OUDEYER, P.-Y., AND TEODORESCU, L. Aces: Generating a diversity of challenging programming puzzles with autotelic generative models. *Advances in Neural Information Processing Systems* 37 (2024), 67627–67662.
- [73] PRADEL, M. Testora: Using natural language intent to detect behavioral regressions. *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE)* (2026).

- [74] PREKOPCSAK, Z., MAKRAI, G., HENK, T., AND GASPAPANEK, C. Radoop: Analyzing big data with rapidminer and hadoop. In *Proceedings of the 2nd RapidMiner Community Meeting and Conference, RCOMM* (2011), vol. 2011, Citeseer, pp. 1–12.
- [75] PYTHON BABEL/BABEL. Fix extracted lineno with nested calls. <https://github.com/python-babel/babel/issues/1126>.
- [76] PYTHON BABEL/BABEL. Use ast instead of eval for string extraction. <https://github.com/python-babel/babel/issues/915>.
- [77] QI, Z., LUO, H., HUANG, X., ZHAO, Z., JIANG, Y., FAN, X., LAKKARAJU, H., AND GLASS, J. Quantifying generalization complexity for large language models. *arXiv preprint arXiv:2410.01769* (2024).
- [78] QWENLM. What's the cutoff date of codeqwen1.5-7b? · issue #78 · qwenlm/qwen2.5-coder.
- [79] ROBERTS, M., THAKUR, H., HERLIHY, C., WHITE, C., AND DOOLEY, S. To the cutoff... and beyond? a longitudinal perspective on llm data contamination. In *The Twelfth International Conference on Learning Representations* (2023).
- [80] RUAN, H., ZHANG, Y., AND ROYCHOUDHURY, A. Specrover: Code intent extraction via llms. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)* (2025), IEEE, pp. 963–974.
- [81] RYAN, G., JAIN, S., SHANG, M., WANG, S., MA, X., RAMANATHAN, M. K., AND RAY, B. Code-aware prompting: A study of coverage-guided test generation in regression setting using llm. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 951–971.
- [82] SAHA, S., SARKER, L., SHAFIUZZAMAN, M., SHOU, C., LI, A., SANKARAN, G., AND BULTAN, T. Rare path guided fuzzing. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis* (2023), pp. 1295–1306.
- [83] SATPUTE, A., GIEBING, N., GREINER-PETTER, A., SCHUBOTZ, M., TESCHKE, O., AIZAWA, A., AND GIPP, B. Can llms master math? investigating large language models on math stack exchange. In *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval* (2024), pp. 2316–2320.
- [84] SCHÄFER, M., NADI, S., EGHBALI, A., AND TIP, F. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering* (2023).
- [85] SHAO, Z., WANG, P., ZHU, Q., XU, R., SONG, J., BI, X., ZHANG, H., ZHANG, M., LI, Y., WU, Y., ET AL. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300* (2024).
- [86] SIDDIQ, M. L., DA SILVA SANTOS, J. C., TANVIR, R. H., ULFAT, N., AL RIFAT, F., AND CARVALHO LOPES, V. Using large language models to generate junit tests: An empirical study. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering* (2024), pp. 313–322.
- [87] SOBANIA, D., BRIESCHI, M., HANNA, C., AND PETKE, J. An analysis of the automatic bug fixing performance of chatgpt. In *2023 IEEE/ACM International Workshop on Automated Program Repair (APR)* (2023), IEEE, pp. 23–30.
- [88] TAMBON, F., MORADI-DAKHEL, A., NIKANJAM, A., KHOMH, F., DESMARAIS, M. C., AND ANTONIOL, G. Bugs in large language models generated code: An empirical study. *Empirical Software Engineering* 30, 3 (2025), 65.
- [89] TAMKIN, A., HANDA, K., SHRESTHA, A., AND GOODMAN, N. Task ambiguity in humans and language models. In *The Eleventh International Conference on Learning Representations*.
- [90] TANG, L., LIU, J., LIU, Z., YANG, X., AND BAO, L. Llm4szz: Enhancing szz algorithm with context-enhanced assessment on large language models. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 343–365.
- [91] TANG, Y., LIU, Z., ZHOU, Z., AND LUO, X. Chatgpt vs sbst: A comparative assessment of unit test suite generation. *IEEE Transactions on Software Engineering* (2024).
- [92] THESKUMAR/PYTHON DOTENV. Some more s/python-dotenv/python-dotenv/. <https://github.com/theskumar/python-dotenv/commit/9acba4af31757e99e2d6e6700de621ee8f9b98ae>.
- [93] TIAN, Y., YAN, W., YANG, Q., ZHAO, X., CHEN, Q., WANG, W., LUO, Z., MA, L., AND SONG, D. Codehalu: Investigating code hallucinations in llms via execution-based verification. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2025), vol. 39, pp. 25300–25308.
- [94] TIAN, Y., ZHANG, Y., STOL, K.-J., JIANG, L., AND LIU, H. What makes a good commit message? In *Proceedings of the 44th International Conference on Software Engineering* (2022), pp. 2389–2401.
- [95] TIAN, Z., CHEN, J., AND ZHANG, X. Fixing large language models' specification misunderstanding for better code generation. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)* (2025), IEEE Computer Society, pp. 645–645.
- [96] TSIMPOURLAS, F., RAJAN, A., AND ALLAMANIS, M. Supervised learning over test executions as a test oracle. In *Proceedings of the 36th Annual ACM Symposium on Applied Computing* (2021), pp. 1521–1531.
- [97] VALENTIN, T., MADADI, A., SAPIA, G., AND BÖHME, M. Estimating correctness without oracles in llm-based code generation. *The 40th Annual AAAI Conference on Artificial Intelligence* (2026).
- [98] WANG, J., HUANG, Y., WANG, S., AND WANG, Q. Find bugs in static bug finders. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension* (2022), pp. 516–527.
- [99] WANG, J., YU, L., AND LUO, X. Llmif: Augmented large language model for fuzzing iot devices. In *2024 IEEE Symposium on Security and Privacy (SP)* (2024), IEEE, pp. 881–896.
- [100] WANG, Q., HUANG, J.-F., AND GONG, Y.-Z. Research on loop path selection in coverage testing. In *2013 Fourth World Congress on Software Engineering* (2013), IEEE, pp. 99–104.
- [101] WANG, W., YANG, C., WANG, Z., HUANG, Y., CHU, Z., SONG, D., ZHANG, L., CHEN, A. R., AND MA, L. Testeval: Benchmarking large language models for test case generation. *arXiv preprint arXiv:2406.04531* (2024).

- [102] WANG, Y., HE, K., FU, D., GONGQUE, Z., XU, H., CHEN, Y., WANG, Z., FU, Y., DONG, G., DIAO, M., ET AL. How do your code llms perform? empowering code instruction tuning with really good data. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing* (2024), pp. 14027–14043.
- [103] WANG, Z., YAN, M., CHEN, J., LIU, S., AND ZHANG, D. Deep learning library testing via effective model generation. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (2020), pp. 788–799.
- [104] WATSON, C., TUFANO, M., MORAN, K., BAVOTA, G., AND POSHYVANYK, D. On learning meaningful assert statements for unit test cases. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering* (2020), pp. 1398–1409.
- [105] WEN, C., CAO, J., SU, J., XU, Z., QIN, S., HE, M., LI, H., CHEUNG, S.-C., AND TIAN, C. Enchanting program specification synthesis by large language models using static analysis and program verification. In *International Conference on Computer Aided Verification* (2024), Springer, pp. 302–328.
- [106] WEN, M., CHEN, J., WU, R., HAO, D., AND CHEUNG, S.-C. Context-aware patch generation for better automated program repair. In *Proceedings of the 40th international conference on software engineering* (2018), pp. 1–11.
- [107] XIA, C. S., DENG, Y., AND ZHANG, L. Top leaderboard ranking= top coding proficiency, always? evoeval: Evolving coding benchmarks via llm. *arXiv preprint arXiv:2403.19114* (2024).
- [108] XIA, C. S., PALTENGLI, M., LE TIAN, J., PRADEL, M., AND ZHANG, L. Fuzz4all: Universal fuzzing with large language models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (2024), pp. 1–13.
- [109] XIA, S. Release evoeval v0.1.0 · evo-eval/evoeval.
- [110] XIONG, Y., XU, M., SU, T., SUN, J., WANG, J., WEN, H., PU, G., HE, J., AND SU, Z. An empirical study of functional bugs in android apps. In *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis* (2023), pp. 1319–1331.
- [111] XU, F., LIN, Q., HAN, J., ZHAO, T., LIU, J., AND CAMBRIA, E. Are large language models really good logical reasoners? a comprehensive evaluation and beyond. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [112] YAMAUCHI, K., YANG, J., HOTTA, K., HIGO, Y., AND KUSUMOTO, S. Clustering commits for understanding the intents of implementation. In *2014 IEEE international conference on software maintenance and evolution* (2014), IEEE, pp. 406–410.
- [113] YANG, C., CHEN, J., LIN, B., ZHOU, J., AND WANG, Z. Enhancing llm-based test generation for hard-to-cover branches via program analysis. *arXiv preprint arXiv:2404.04966* (2024).
- [114] YANG, C., ZHAO, Z., AND ZHANG, L. Kernelgpt: Enhanced kernel fuzzing via large language models. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (2025), pp. 560–573.
- [115] YANG, J., WANG, Y., LOU, Y., WEN, M., AND ZHANG, L. A large-scale empirical review of patch correctness checking approaches. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (2023), pp. 1203–1215.
- [116] YE, T., ZHANG, L., WANG, L., AND LI, X. An empirical study on detecting and fixing buffer overflow bugs. In *2016 IEEE International Conference on Software Testing, Verification and Validation (ICST)* (2016), IEEE, pp. 91–101.
- [117] YU, H., SHEN, B., RAN, D., ZHANG, J., ZHANG, Q., MA, Y., LIANG, G., LI, Y., WANG, Q., AND XIE, T. Codereval: A benchmark of pragmatic code generation with generative pre-trained models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering* (2024), pp. 1–12.
- [118] YU, X., LIU, L., HU, X., KEUNG, J., XIA, X., AND LO, D. Practitioners’ expectations on automated test generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (2024), pp. 1618–1630.
- [119] YUAN, Z., LIU, M., DING, S., WANG, K., CHEN, Y., PENG, X., AND LOU, Y. Evaluating and improving chatgpt for unit test generation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1703–1726.
- [120] ZHANG, F., CHEN, B., ZHANG, Y., KEUNG, J., LIU, J., ZAN, D., MAO, Y., LOU, J.-G., AND CHEN, W. Repocoder: Repository-level code completion through iterative retrieval and generation. In *The 2023 Conference on Empirical Methods in Natural Language Processing*.
- [121] ZHANG, L., HE, S., ZHANG, C., KANG, Y., LI, B., XIE, C., WANG, J., WANG, M., HUANG, Y., FU, S., ET AL. Swe-bench goes live! *arXiv preprint arXiv:2505.23419* (2025).
- [122] ZHANG, L., HE, S., ZHANG, C., KANG, Y., LI, B., XIE, C., WANG, J., WANG, M., HUANG, Y., FU, S., NALLIPOGU, E., LIN, Q., DANG, Y., RAJMOHAN, S., AND ZHANG, D. Swe-bench goes live! *arXiv preprint arXiv:2505.23419* (2025).
- [123] ZHONG, H., AND SU, Z. An empirical study on real bug fixes. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering* (2015), vol. 1, IEEE, pp. 913–923.
- [124] ZHUO, T. Y., VU, M. C., CHIM, J., HU, H., YU, W., WIDYASARI, R., YUSUF, I. N. B., ZHAN, H., HE, J., PAUL, I., ET AL. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877* (2024).